

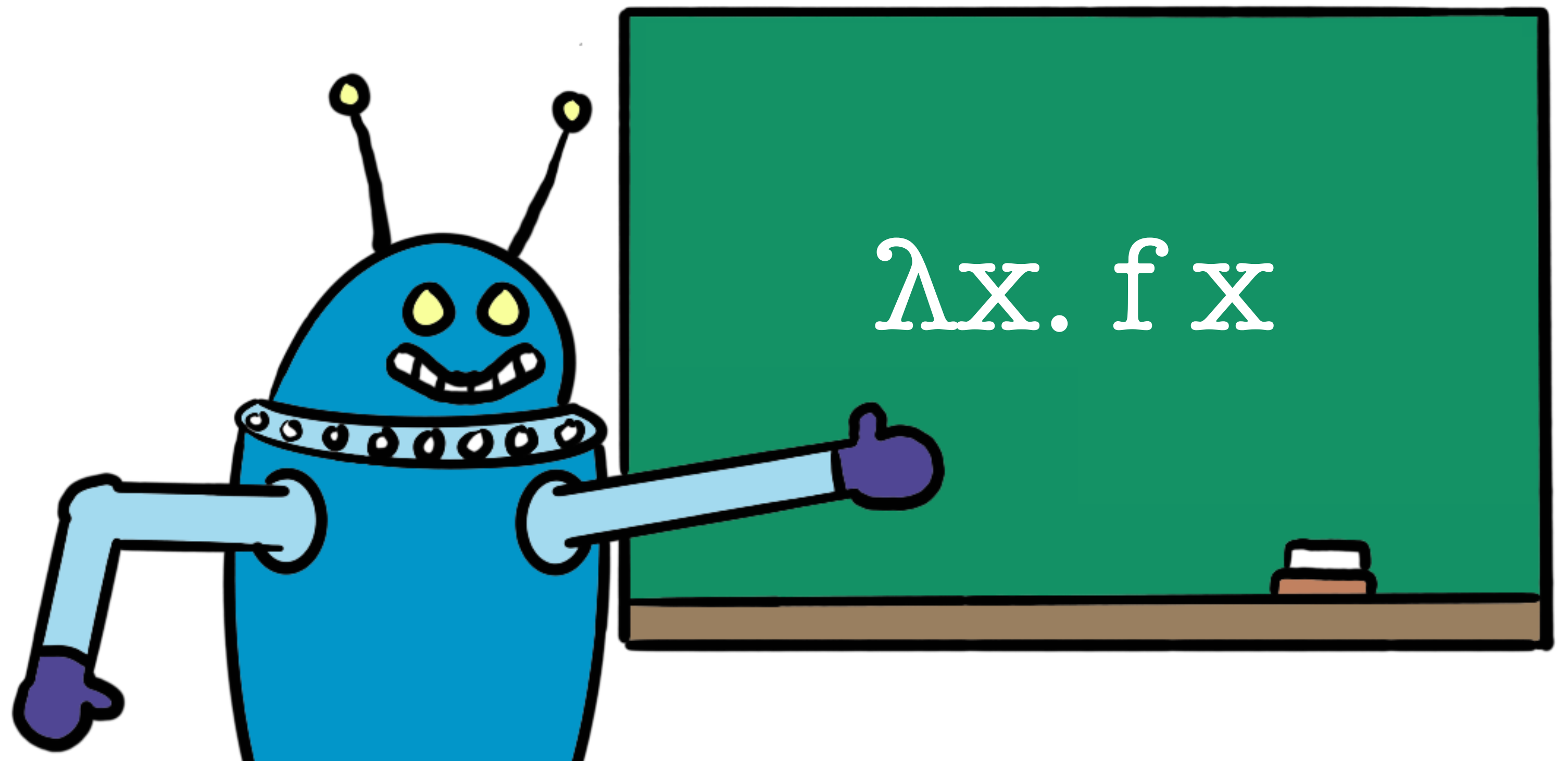
Provers and Solvers

Lecture 3: λ -Superposition

Jasmin Blanchette

LMU Munich

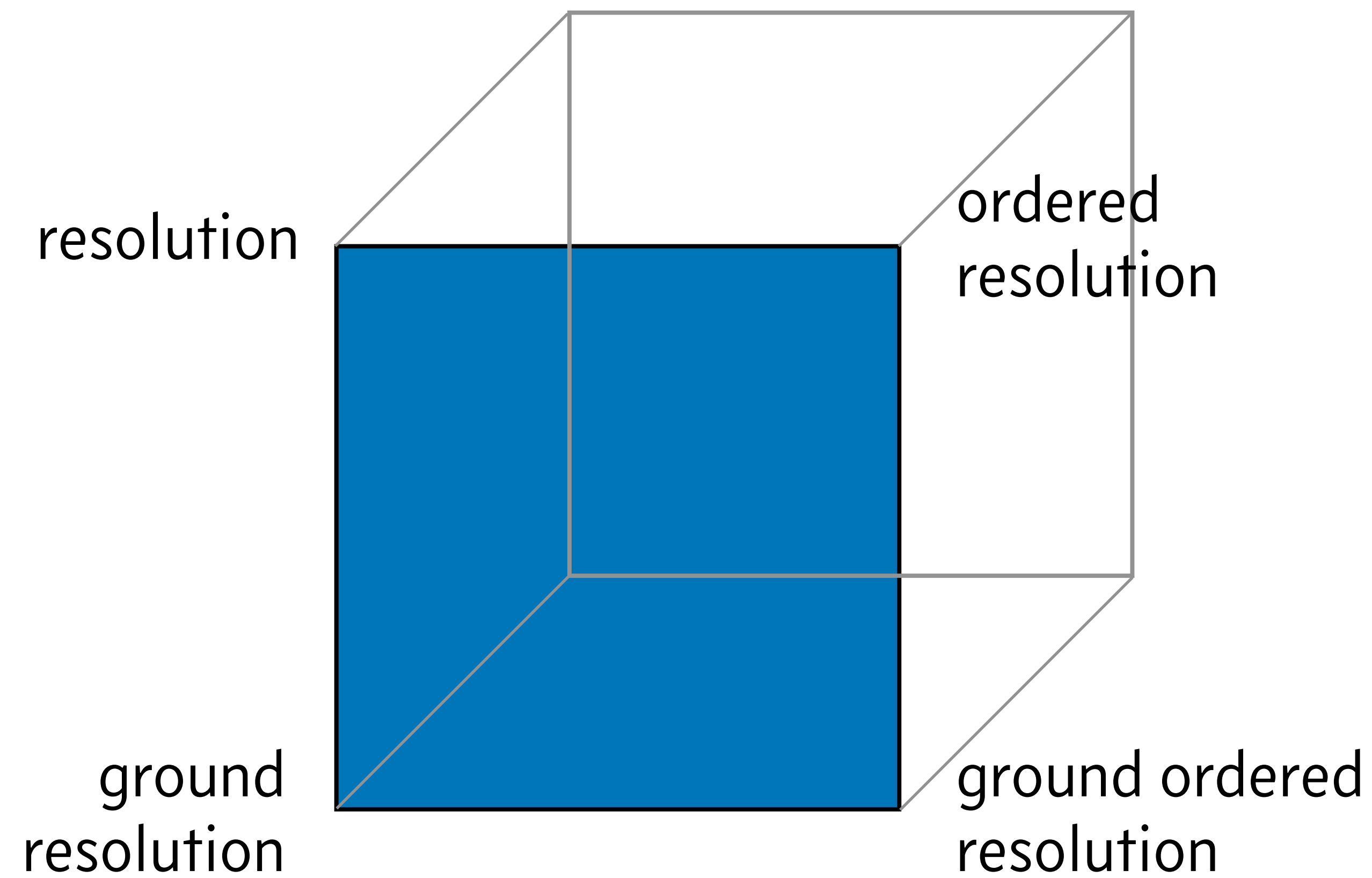
Partly based on slides by
Alexander Bentkamp



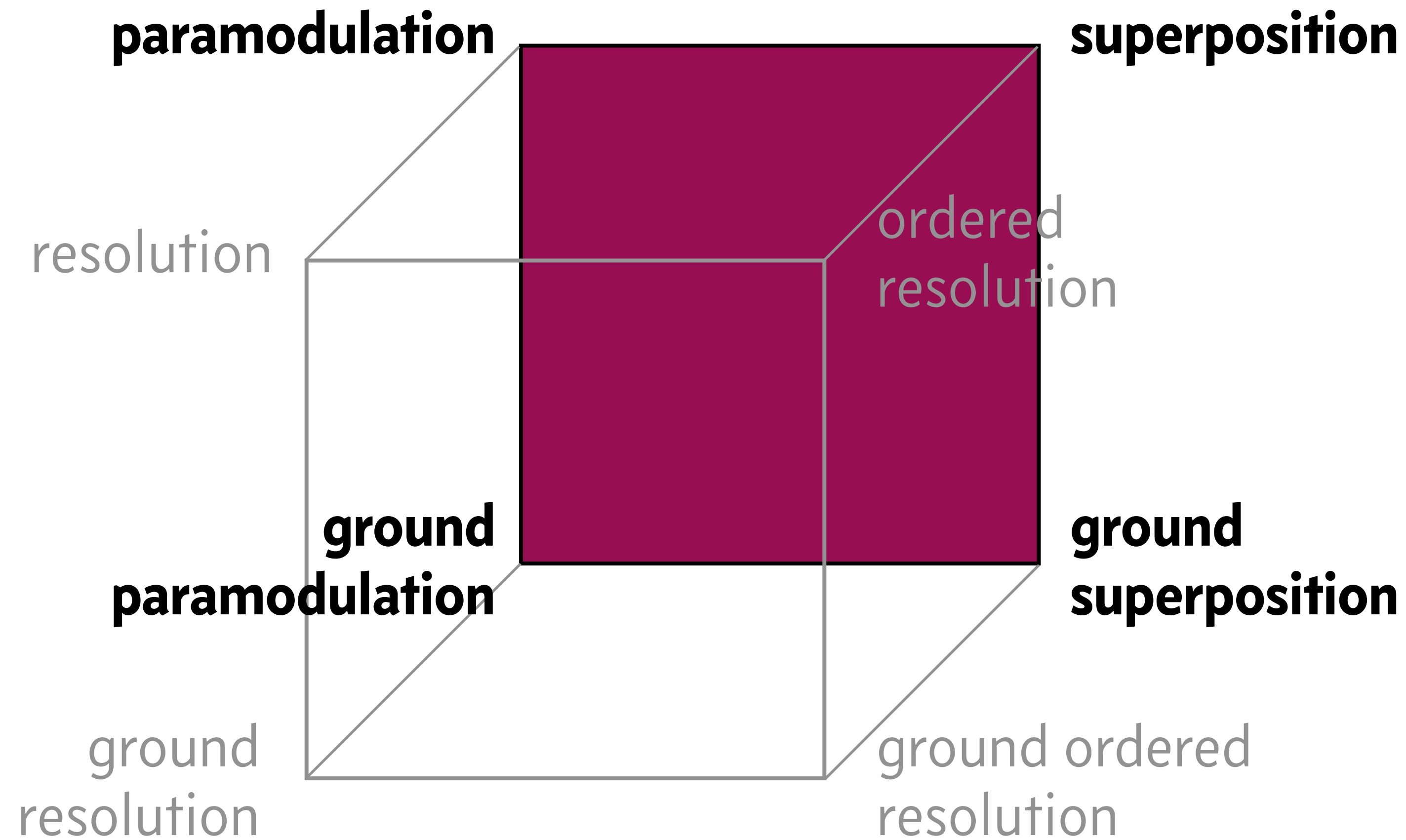
Outline of These Lectures

1. Resolution
2. Superposition
- 3. λ -Superposition**
4. AVATAR

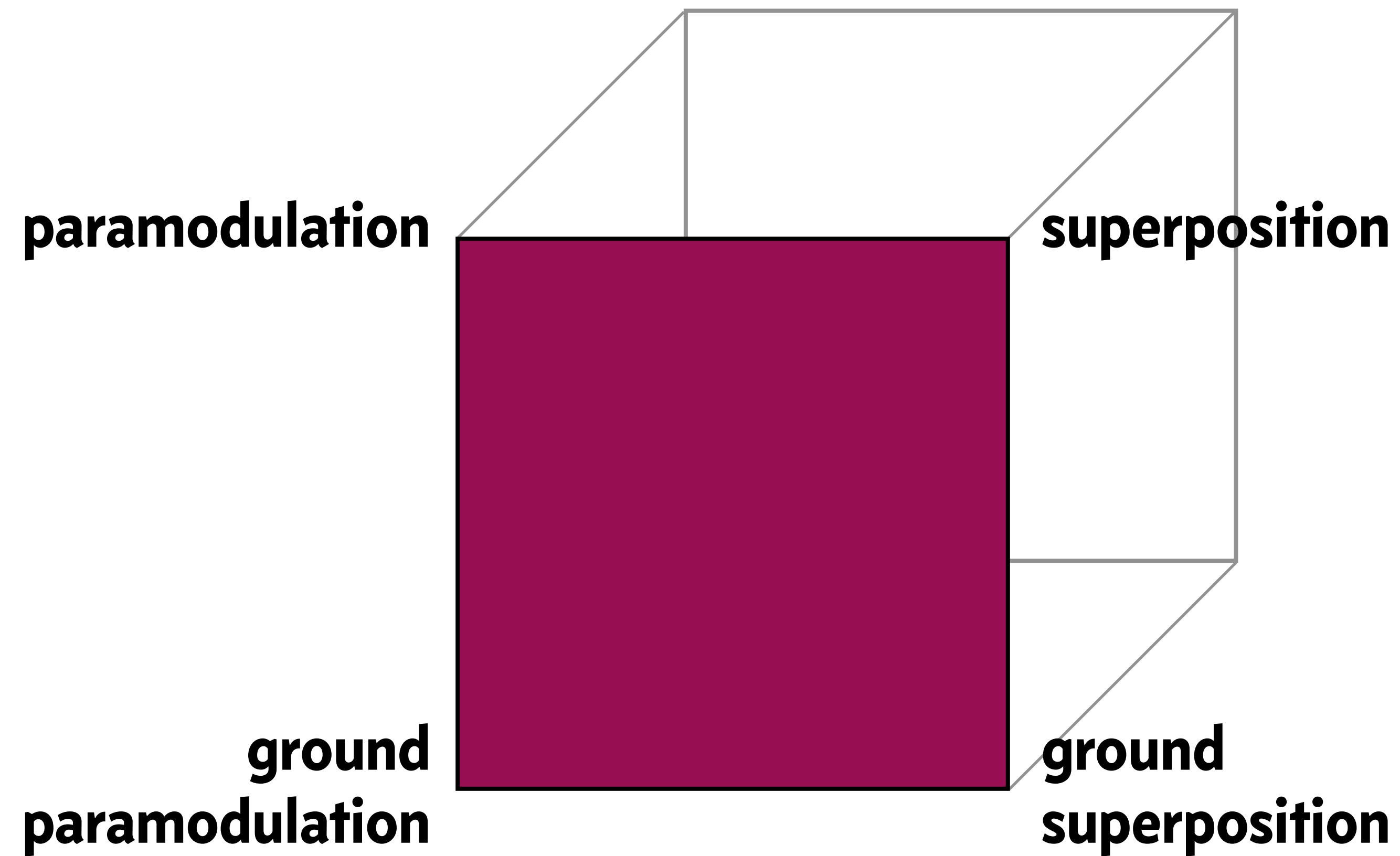
Relations between the Calculi



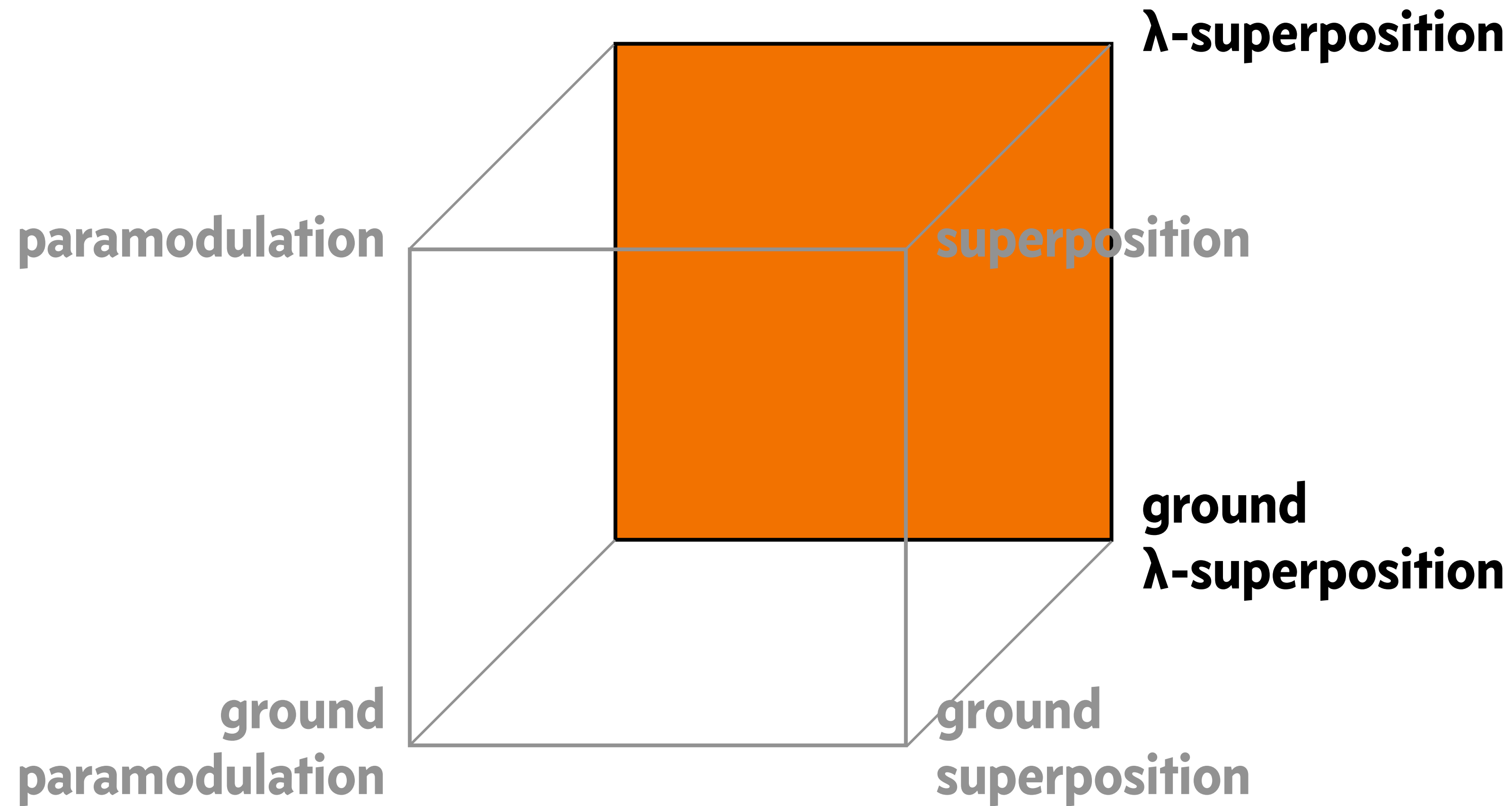
Relations between the Calculi



Relations between the Calculi



Relations between the Calculi



A Higher-Order Proof Goal

Proof assistant

show

$$\left(\sum_{i=1}^n i^2 + 2i + 1\right) = \left(\sum_{i=1}^n i^2\right) + \left(\sum_{i=1}^n 2i\right) + \left(\sum_{i=1}^n 1\right)$$

Find Proof

A Higher-Order Proof Goal

Proof assistant

show

$$\left(\sum_{i=1}^n i^2 + 2i + 1\right) = \left(\sum_{i=1}^n i^2\right) + \left(\sum_{i=1}^n 2i\right) + \left(\sum_{i=1}^n 1\right)$$

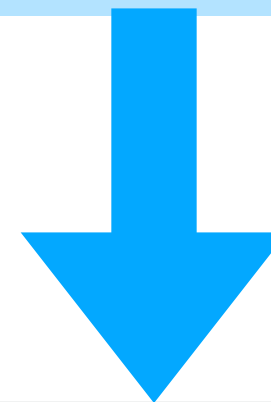
Find Proof

Lost in Translation

$$\left(\sum_{i=1}^n i^2 + 2i + 1\right) = \left(\sum_{i=1}^n i^2\right) + \left(\sum_{i=1}^n 2i\right) + \left(\sum_{i=1}^n 1\right)$$

Lost in Translation

$$\left(\sum_{i=1}^n i^2 + 2i + 1\right) = \left(\sum_{i=1}^n i^2\right) + \left(\sum_{i=1}^n 2i\right) + \left(\sum_{i=1}^n 1\right)$$



```
sum(1, n, C(B(plus, S(B(plus, C(power, 2)), app(times, 2))), 1))  
= app(app(plus, app(app(plus, sum(1, n, C(power, 2))),  
    sum(1, n, app(times, 2)))), sum(1, n, K(1)))
```

Design Goal for λ -Superposition

A sound, complete, **graceful generalization**
of first-order superposition

Syntax of Higher-Order Logic

Given a **signature**, consisting of

- atomic types (e.g., `bool`, `nat`)
- symbols (e.g., `1`, `·`, `gcd`) declared with their types

Syntax of Higher-Order Logic

Given a **signature**, consisting of

- atomic types (e.g., `bool`, `nat`)
- symbols (e.g., `1`, `·`, `gcd`) declared with their types

The **terms** are defined by these rules:

- A variable x declared with type τ is a term of type τ
- A symbol f declared with type τ is a term of type τ
- If t has type τ , an abstraction $\lambda x : \sigma. t$ is a term of type $\sigma \rightarrow \tau$
- If t has type $\sigma \rightarrow \tau$ and t' has type σ , an application $t t'$ is a term of type τ

Syntax of Higher-Order Logic

Given a **signature**, consisting of

- atomic types (e.g., `bool`, `nat`)
- symbols (e.g., `1`, `·`, `gcd`) declared with their types

Formulas are terms of type `bool`

The **terms** are defined by these rules:

- A variable x declared with type τ is a term of type τ
- A symbol f declared with type τ is a term of type τ
- If t has type τ , an abstraction $\lambda x : \sigma. t$ is a term of type $\sigma \rightarrow \tau$
- If t has type $\sigma \rightarrow \tau$ and t' has type σ , an application $t t'$ is a term of type τ

Examples: Higher-Order Terms

$g\ a\ x$

$f\ (g\ a\ x)\ b\ y$

$p\ f\ (f\ a)\ (f\ a\ b)$

$\lambda x. \lambda y. g\ y\ x$

$1 + 2$

$\alpha\beta\eta$ -Equivalence

Terms are considered equal up to the following three rules:

$$(\alpha) \ (\lambda x. t(x)) = (\lambda y. t(y))$$

$$(\beta) \ (\lambda x. t(x)) u = t(u)$$

$$(\eta) \ (\lambda x. t\ x) = t \quad \text{if } x \text{ does not occur free in } t$$

Examples: $\alpha\beta\eta$ -Equivalence

$$g (\lambda x. f x x) = g (\lambda z. f z z)$$

$$(\lambda x. f x x) a = f a a$$

$$g (f a a) = g (\lambda y. f a a y)$$

Syntax of Clausal Higher-Order Logic

The **atoms** are defined by this rule:

- If t_1 and t_2 are terms, then $t_1 = t_2$ (viewed as an unordered pair) is an atom

The **literals** are defined by this rule:

- $t_1 = t_2$ and $t_1 \neq t_2$ are literals

The **clauses** are defined by this rule:

- If $L_1, \dots, L_n$ are literals, then $L_1 \vee \dots \vee L_n$ (viewed as a multiset) is a clause

We write $\perp$ if $n = 0$

Syntax of Clausal Higher-Order Logic

The **atoms** are defined by this rule:

- If t_1 and t_2 are terms, then $t_1 = t_2$ (viewed as an unordered pair) is an atom

The **literals** are defined by this rule:

- $t_1 = t_2$ and $t_1 \neq t_2$ are literals

The **clauses** are defined by this rule:

- If $L_1, \dots, L_n$ are literals, then $L_1 \vee \dots \vee L_n$ (viewed as a multiset) is a clause

We write $\perp$ if $n = 0$



- $\top, \wedge, \Rightarrow, \forall, \exists$ can appear in terms
- Variables are understood as "for all"

Example: Summations

Goal:

$$\left(\sum_{i=1}^n i^2 + 2i + 1 \right) = \left(\sum_{i=1}^n i^2 \right) + \left(\sum_{i=1}^n 2i \right) + \left(\sum_{i=1}^n 1 \right)$$

Distributivity lemma:

$$\forall f, g. \sum_{i=1}^n (f i + g i) = \sum_{i=1}^n f i + \sum_{i=1}^n g i$$

Proof idea:

$$\left(\sum_{i=1}^n i^2 + 2i + 1 \right) = \sum_{i=1}^n (i^2 + 2i) + \sum_{i=1}^n 1 = \left(\sum_{i=1}^n i^2 \right) + \left(\sum_{i=1}^n 2i \right) + \left(\sum_{i=1}^n 1 \right)$$

Example: Summations

Goal:

$$\left(\sum_{i=1}^n i^2 + 2i + 1 \right) = \left(\sum_{i=1}^n i^2 \right) + \left(\sum_{i=1}^n 2i \right) + \left(\sum_{i=1}^n 1 \right)$$

$$\text{sum } 1 \text{ n } (\lambda i. i^2 + 2i + 1) = \text{sum } 1 \text{ n } (\lambda i. i^2) + \text{sum } 1 \text{ n } (\lambda i. 2i) + \text{sum } 1 \text{ n } (\lambda i. 1)$$

Distributivity lemma:

$$\forall f, g. \sum_{i=1}^n (f i + g i) = \sum_{i=1}^n f i + \sum_{i=1}^n g i$$

$$\text{sum } 1 \text{ n } (\lambda i. f i + g i) = \text{sum } 1 \text{ n } (\lambda i. f i) + \text{sum } 1 \text{ n } (\lambda i. g i)$$

Proof idea:

$$\left(\sum_{i=1}^n i^2 + 2i + 1 \right) = \sum_{i=1}^n (i^2 + 2i) + \sum_{i=1}^n 1 = \left(\sum_{i=1}^n i^2 \right) + \left(\sum_{i=1}^n 2i \right) + \left(\sum_{i=1}^n 1 \right)$$

Example: Summations

$$\begin{aligned} & \sum_{i=1}^n (\lambda_i \cdot f_i + g_i) \\ = & \sum_{i=1}^n (\lambda_i \cdot f_i) + \sum_{i=1}^n (\lambda_i \cdot g_i) \end{aligned}$$

$$\begin{aligned} & \sum_{i=1}^n (\lambda_i \cdot i^2 + 2i + 1) \\ \neq & \sum_{i=1}^n (\lambda_i \cdot i^2) + \sum_{i=1}^n (\lambda_i \cdot 2i) + \sum_{i=1}^n (\lambda_i \cdot 1) \end{aligned}$$

Example: Summations

$$\begin{aligned} & \sum_{i=1}^n (\lambda i \cdot f i + g i) \\ &= \sum_{i=1}^n (\lambda i \cdot f i) + \sum_{i=1}^n (\lambda i \cdot g i) \end{aligned}$$

$$\begin{aligned} & \sum_{i=1}^n (\lambda i \cdot i^2 + 2i + 1) \\ & \neq \sum_{i=1}^n (\lambda i \cdot i^2) + \sum_{i=1}^n (\lambda i \cdot 2i) + \sum_{i=1}^n (\lambda i \cdot 1) \end{aligned}$$


$$\begin{aligned} & \sum_{i=1}^n (\lambda i \cdot i^2 + 2i + 1) \\ & \neq \sum_{i=1}^n (\lambda i \cdot i^2 + 2i) + \sum_{i=1}^n (\lambda i \cdot 1) \end{aligned}$$

Example: Summations

$$\sum_{i=1}^n (\lambda i \cdot fi + gi) \\ = \sum_{i=1}^n (\lambda i \cdot fi) + \sum_{i=1}^n (\lambda i \cdot gi)$$

$$\sum_{i=1}^n (\lambda i \cdot i^2 + 2i + 1) \\ \neq \sum_{i=1}^n (\lambda i \cdot i^2) + \sum_{i=1}^n (\lambda i \cdot 2i) + \sum_{i=1}^n (\lambda i \cdot 1)$$

$$\sum_{i=1}^n (\lambda i \cdot i^2 + 2i + 1) \\ \neq \sum_{i=1}^n (\lambda i \cdot i^2 + 2i) + \sum_{i=1}^n (\lambda i \cdot 1)$$

Example: Summations

$$\begin{aligned} & \text{sum } 1 \text{ n } (\lambda i. fi + gi) \\ = & \text{sum } 1 \text{ n } (\lambda i. fi) + \text{sum } 1 \text{ n } (\lambda i. gi) \end{aligned}$$

$$\begin{aligned} & \text{sum } 1 \text{ n } (\lambda i. i^2 + 2i + 1) \\ \neq & \text{sum } 1 \text{ n } (\lambda i. i^2) + \text{sum } 1 \text{ n } (\lambda i. 2i) + \text{sum } 1 \text{ n } (\lambda i. 1) \end{aligned}$$

$$\begin{aligned} & \text{sum } 1 \text{ n } (\lambda i. i^2 + 2i + 1) \\ \neq & \text{sum } 1 \text{ n } (\lambda i. i^2 + 2i) + \text{sum } 1 \text{ n } (\lambda i. 1) \end{aligned}$$

Example: Summations

$$\begin{aligned} & \sum_{i=1}^n (\lambda_i \cdot f_i + g_i) \\ = & \sum_{i=1}^n (\lambda_i \cdot f_i) + \sum_{i=1}^n (\lambda_i \cdot g_i) \end{aligned}$$

$$\begin{aligned} & \sum_{i=1}^n (\lambda_i \cdot i^2 + 2i + 1) \\ \neq & \sum_{i=1}^n (\lambda_i \cdot i^2) + \sum_{i=1}^n (\lambda_i \cdot 2i) + \sum_{i=1}^n (\lambda_i \cdot 1) \end{aligned}$$

$$\begin{aligned} & \sum_{i=1}^n (\lambda_i \cdot i^2 + 2i + 1) \\ \neq & \sum_{i=1}^n (\lambda_i \cdot i^2 + 2i) + \sum_{i=1}^n (\lambda_i \cdot 1) \end{aligned}$$

Example: Summations

$$\begin{aligned} & \text{sum } 1 \text{ n } (\lambda i. fi + gi) \\ &= \text{sum } 1 \text{ n } (\lambda i. fi) + \text{sum } 1 \text{ n } (\lambda i. gi) \end{aligned}$$

$$\begin{aligned} & \text{sum } 1 \text{ n } (\lambda i. i^2 + 2i + 1) \\ & \neq \text{sum } 1 \text{ n } (\lambda i. i^2) + \text{sum } 1 \text{ n } (\lambda i. 2i) + \text{sum } 1 \text{ n } (\lambda i. 1) \end{aligned}$$

$$\begin{aligned} & \text{sum } 1 \text{ n } (\lambda i. i^2 + 2i + 1) \\ & \neq \text{sum } 1 \text{ n } (\lambda i. i^2 + 2i) + \text{sum } 1 \text{ n } (\lambda i. 1) \end{aligned}$$

$$\begin{aligned} & \text{sum } 1 \text{ n } (\lambda i. i^2 + 2i + 1) \\ & \neq \text{sum } 1 \text{ n } (\lambda i. i^2 + 2i + 1) \end{aligned}$$

Example: Summations

$$\begin{aligned} & \text{sum } 1 \text{ n } (\lambda i. fi + gi) \\ = & \text{sum } 1 \text{ n } (\lambda i. fi) + \text{sum } 1 \text{ n } (\lambda i. gi) \end{aligned}$$

$$\begin{aligned} & \text{sum } 1 \text{ n } (\lambda i. i^2 + 2i + 1) \\ \neq & \text{sum } 1 \text{ n } (\lambda i. i^2) + \text{sum } 1 \text{ n } (\lambda i. 2i) + \text{sum } 1 \text{ n } (\lambda i. 1) \end{aligned}$$

$$\begin{aligned} & \text{sum } 1 \text{ n } (\lambda i. i^2 + 2i + 1) \\ \neq & \text{sum } 1 \text{ n } (\lambda i. i^2 + 2i) + \text{sum } 1 \text{ n } (\lambda i. 1) \end{aligned}$$

$$\begin{aligned} & \text{sum } 1 \text{ n } (\lambda i. i^2 + 2i + 1) \\ \neq & \text{sum } 1 \text{ n } (\lambda i. i^2 + 2i + 1) \end{aligned}$$

Example: Summations

$$\begin{aligned} & \text{sum } 1 \text{ n } (\lambda i. fi + gi) \\ = & \text{sum } 1 \text{ n } (\lambda i. fi) + \text{sum } 1 \text{ n } (\lambda i. gi) \end{aligned}$$

$$\begin{aligned} & \text{sum } 1 \text{ n } (\lambda i. i^2 + 2i + 1) \\ \neq & \text{sum } 1 \text{ n } (\lambda i. i^2) + \text{sum } 1 \text{ n } (\lambda i. 2i) + \text{sum } 1 \text{ n } (\lambda i. 1) \end{aligned}$$

$$\begin{aligned} & \text{sum } 1 \text{ n } (\lambda i. i^2 + 2i + 1) \\ \neq & \text{sum } 1 \text{ n } (\lambda i. i^2 + 2i) + \text{sum } 1 \text{ n } (\lambda i. 1) \end{aligned}$$

$$\begin{aligned} & \text{sum } 1 \text{ n } (\lambda i. i^2 + 2i + 1) \\ \neq & \text{sum } 1 \text{ n } (\lambda i. i^2 + 2i + 1) \end{aligned}$$

Example: Summations

$$\begin{aligned} & \sum_{i=1}^n (\lambda i \cdot f_i + g_i) \\ = & \sum_{i=1}^n (\lambda i \cdot f_i) + \sum_{i=1}^n (\lambda i \cdot g_i) \end{aligned}$$

$$\begin{aligned} & \sum_{i=1}^n (\lambda i \cdot i^2 + 2i + 1) \\ \neq & \sum_{i=1}^n (\lambda i \cdot i^2) + \sum_{i=1}^n (\lambda i \cdot 2i) + \sum_{i=1}^n (\lambda i \cdot 1) \end{aligned}$$

$$\begin{aligned} & \sum_{i=1}^n (\lambda i \cdot i^2 + 2i + 1) \\ \neq & \sum_{i=1}^n (\lambda i \cdot i^2 + 2i) + \sum_{i=1}^n (\lambda i \cdot 1) \end{aligned}$$

$$\begin{aligned} & \sum_{i=1}^n (\lambda i \cdot i^2 + 2i + 1) \\ \neq & \sum_{i=1}^n (\lambda i \cdot i^2 + 2i + 1) \end{aligned}$$

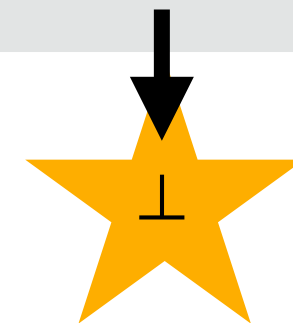
Example: Summations

$$\begin{aligned} & \sum_{i=1}^n (\lambda_i \cdot f_i + g_i) \\ &= \sum_{i=1}^n (\lambda_i \cdot f_i) + \sum_{i=1}^n (\lambda_i \cdot g_i) \end{aligned}$$

$$\begin{aligned} & \sum_{i=1}^n (\lambda_i \cdot i^2 + 2i + 1) \\ &\neq \sum_{i=1}^n (\lambda_i \cdot i^2) + \sum_{i=1}^n (\lambda_i \cdot 2i) + \sum_{i=1}^n (\lambda_i \cdot 1) \end{aligned}$$

$$\begin{aligned} & \sum_{i=1}^n (\lambda_i \cdot i^2 + 2i + 1) \\ &\neq \sum_{i=1}^n (\lambda_i \cdot i^2 + 2i) + \sum_{i=1}^n (\lambda_i \cdot 1) \end{aligned}$$

$$\begin{aligned} & \sum_{i=1}^n (\lambda_i \cdot i^2 + 2i + 1) \\ &\neq \sum_{i=1}^n (\lambda_i \cdot i^2 + 2i + 1) \end{aligned}$$



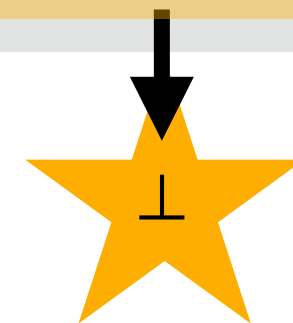
Example: Summations

$$\begin{aligned} & \sum_{i=1}^n (\lambda_i \cdot f_i + g_i) \\ &= \sum_{i=1}^n (\lambda_i \cdot f_i) + \sum_{i=1}^n (\lambda_i \cdot g_i) \end{aligned}$$

$$\begin{aligned} & \sum_{i=1}^n (\lambda_i \cdot i^2 + 2i + 1) \\ &\neq \sum_{i=1}^n (\lambda_i \cdot i^2) + \sum_{i=1}^n (\lambda_i \cdot 2i) + \sum_{i=1}^n (\lambda_i \cdot 1) \end{aligned}$$

$$\begin{aligned} & \sum_{i=1}^n (\lambda_i \cdot i^2 + 2i + 1) \\ &\neq \sum_{i=1}^n (\lambda_i \cdot i^2 + 2i) + \sum_{i=1}^n (\lambda_i \cdot 1) \end{aligned}$$

$$\begin{aligned} & \sum_{i=1}^n (\lambda_i \cdot i^2 + 2i + 1) \\ &\neq \sum_{i=1}^n (\lambda_i \cdot i^2 + 2i + 1) \end{aligned}$$



Example: Summations

$$\begin{aligned} & \sum_{i=1}^n (\lambda_i \cdot f_i + g_i) \\ = & \sum_{i=1}^n (\lambda_i \cdot f_i) + \sum_{i=1}^n (\lambda_i \cdot g_i) \end{aligned}$$

$$\begin{aligned} & \sum_{i=1}^n (\lambda_i \cdot i^2 + 2i + 1) \\ \neq & \sum_{i=1}^n (\lambda_i \cdot i^2) + \sum_{i=1}^n (\lambda_i \cdot 2i) + \sum_{i=1}^n (\lambda_i \cdot 1) \end{aligned}$$

$$\begin{aligned} & \sum_{i=1}^n (\lambda_i \cdot i^2 + 2i + 1) \\ \neq & \sum_{i=1}^n (\lambda_i \cdot i^2 + 2i) + \sum_{i=1}^n (\lambda_i \cdot 1) \end{aligned}$$

$$\begin{aligned} & \sum_{i=1}^n (\lambda_i \cdot i^2 + 2i + 1) \\ \neq & \sum_{i=1}^n (\lambda_i \cdot i^2 + 2i + 1) \end{aligned}$$



Design Challenges for λ -Superposition

1. The term order
2. Unification
3. Booleans

Challenge 1: The Term Order

In first-order superposition:

- A subterm must be smaller than the whole term:
e.g. $c < g(c) < f(g(c))$
- Putting two terms in the same context should preserve the orientation:
e.g. if $b > a$ then $f(b, c) > f(a, c)$

Challenge 1: The Term Order

In first-order superposition:

- A subterm must be smaller than the whole term:
e.g. $c < g(c) < f(g(c))$
- Putting two terms in the same context should preserve the orientation:
e.g. if $b > a$ then $f(b, c) > f(a, c)$

Issue in higher-order:

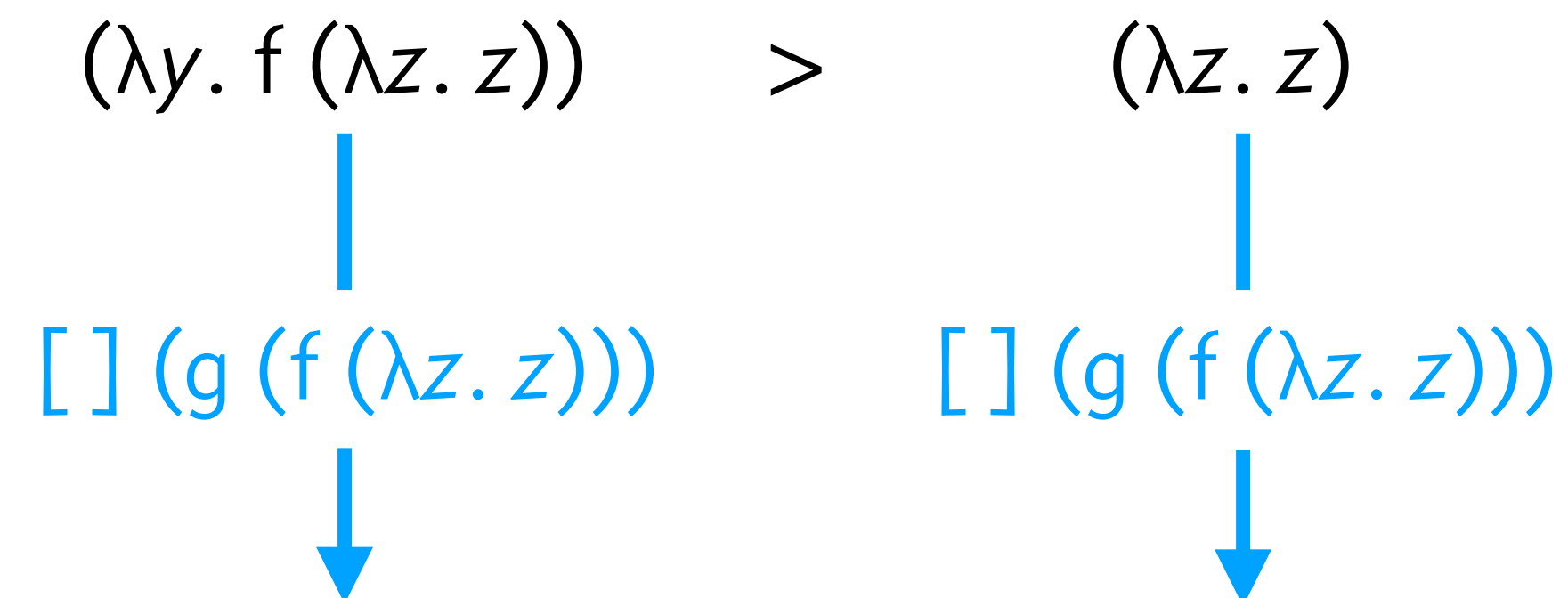
$$(\lambda y. f (\lambda z. z)) \quad > \quad (\lambda z. z)$$

Challenge 1: The Term Order

In first-order superposition:

- A subterm must be smaller than the whole term:
e.g. $c < g(c) < f(g(c))$
- Putting two terms in the same context should preserve the orientation:
e.g. if $b > a$ then $f(b, c) > f(a, c)$

Issue in higher-order:

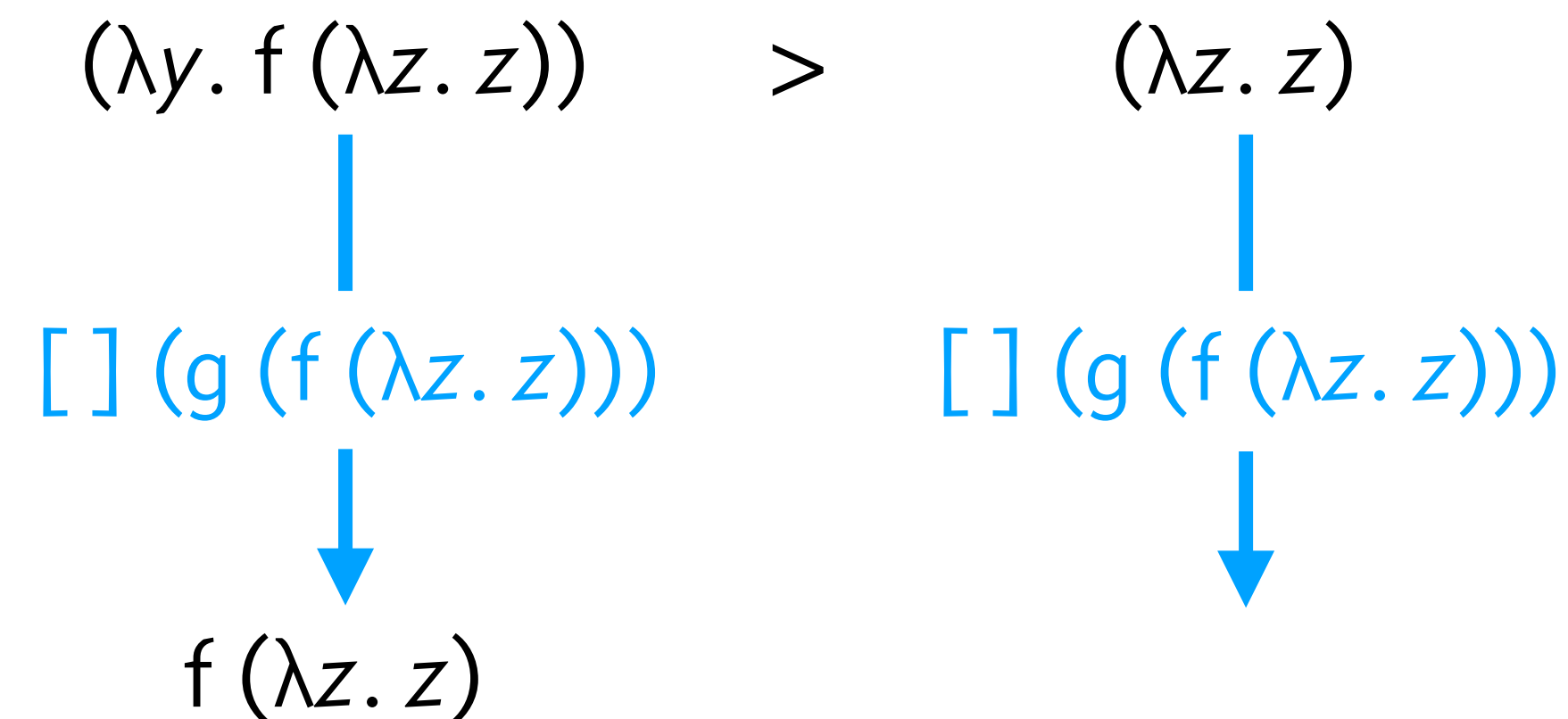


Challenge 1: The Term Order

In first-order superposition:

- A subterm must be smaller than the whole term:
e.g. $c < g(c) < f(g(c))$
- Putting two terms in the same context should preserve the orientation:
e.g. if $b > a$ then $f(b, c) > f(a, c)$

Issue in higher-order:

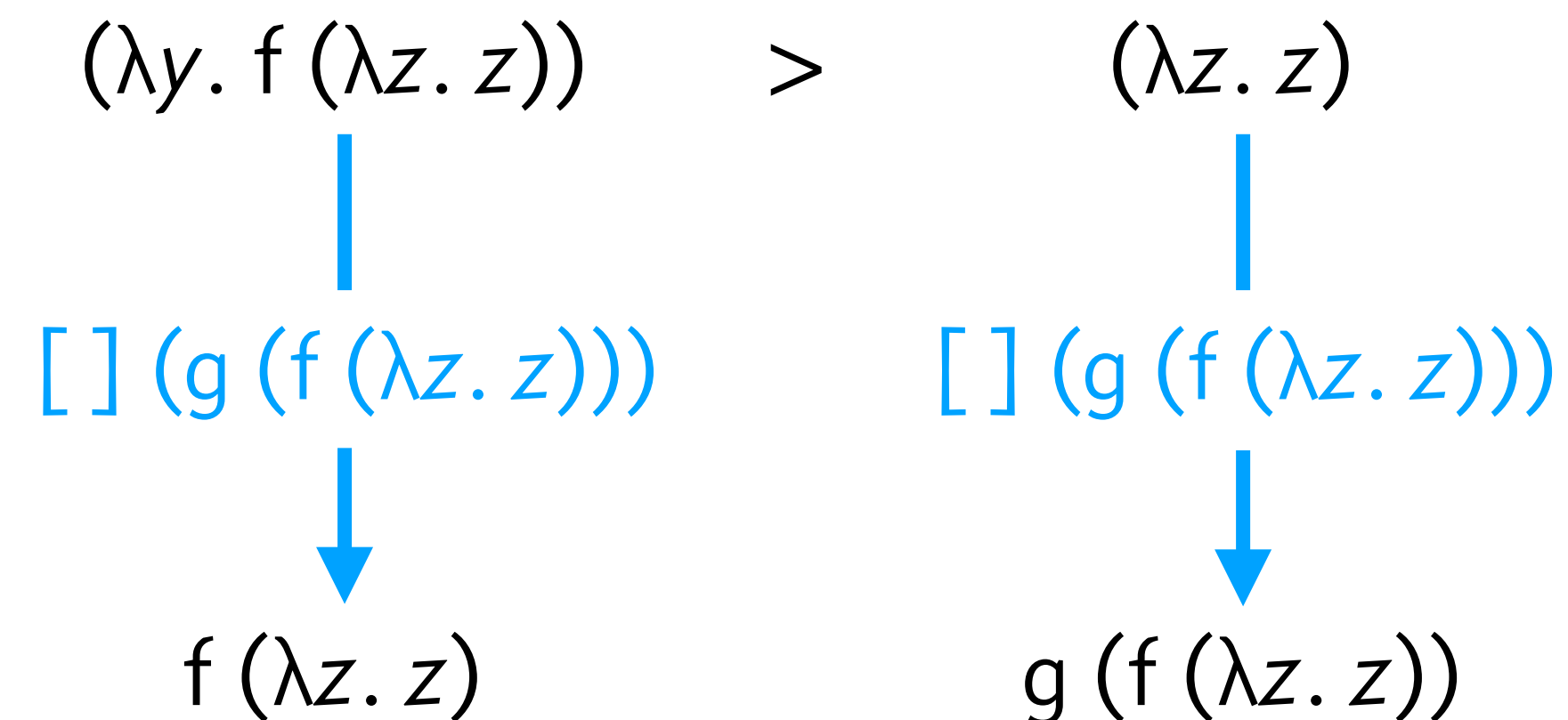


Challenge 1: The Term Order

In first-order superposition:

- A subterm must be smaller than the whole term:
e.g. $c < g(c) < f(g(c))$
- Putting two terms in the same context should preserve the orientation:
e.g. if $b > a$ then $f(b, c) > f(a, c)$

Issue in higher-order:

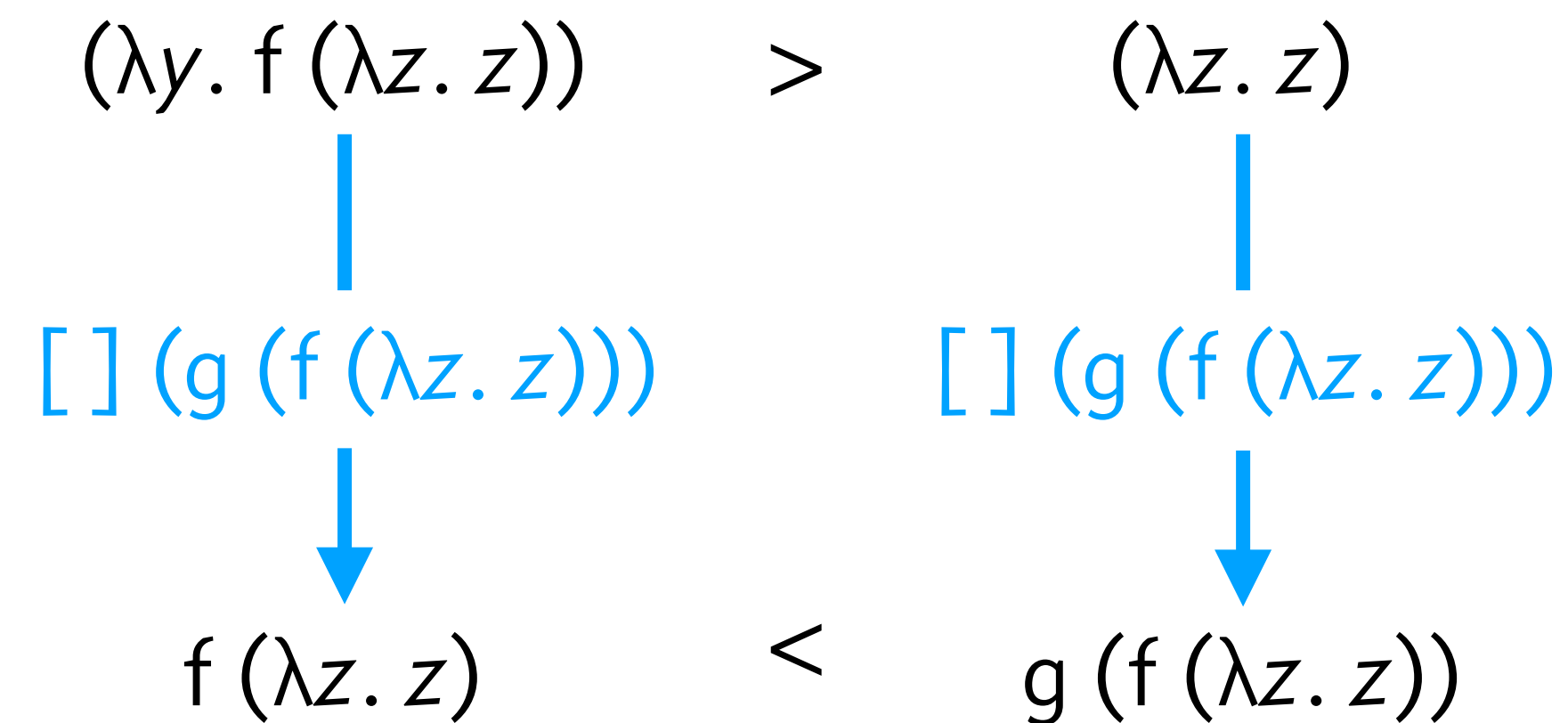


Challenge 1: The Term Order

In first-order superposition:

- A subterm must be smaller than the whole term:
e.g. $c < g(c) < f(g(c))$
- Putting two terms in the same context should preserve the orientation:
e.g. if $b > a$ then $f(b, c) > f(a, c)$

Issue in higher-order:



Challenge 1: The Term Order

In first-order superposition:

- A subterm must be smaller than the whole term:
e.g. $c < g(c) < f(g(c))$
- Putting two terms in the same context should preserve the orientation:
e.g. if $b > a$ then $f(b, c) > f(a, c)$

Issue in higher-order:

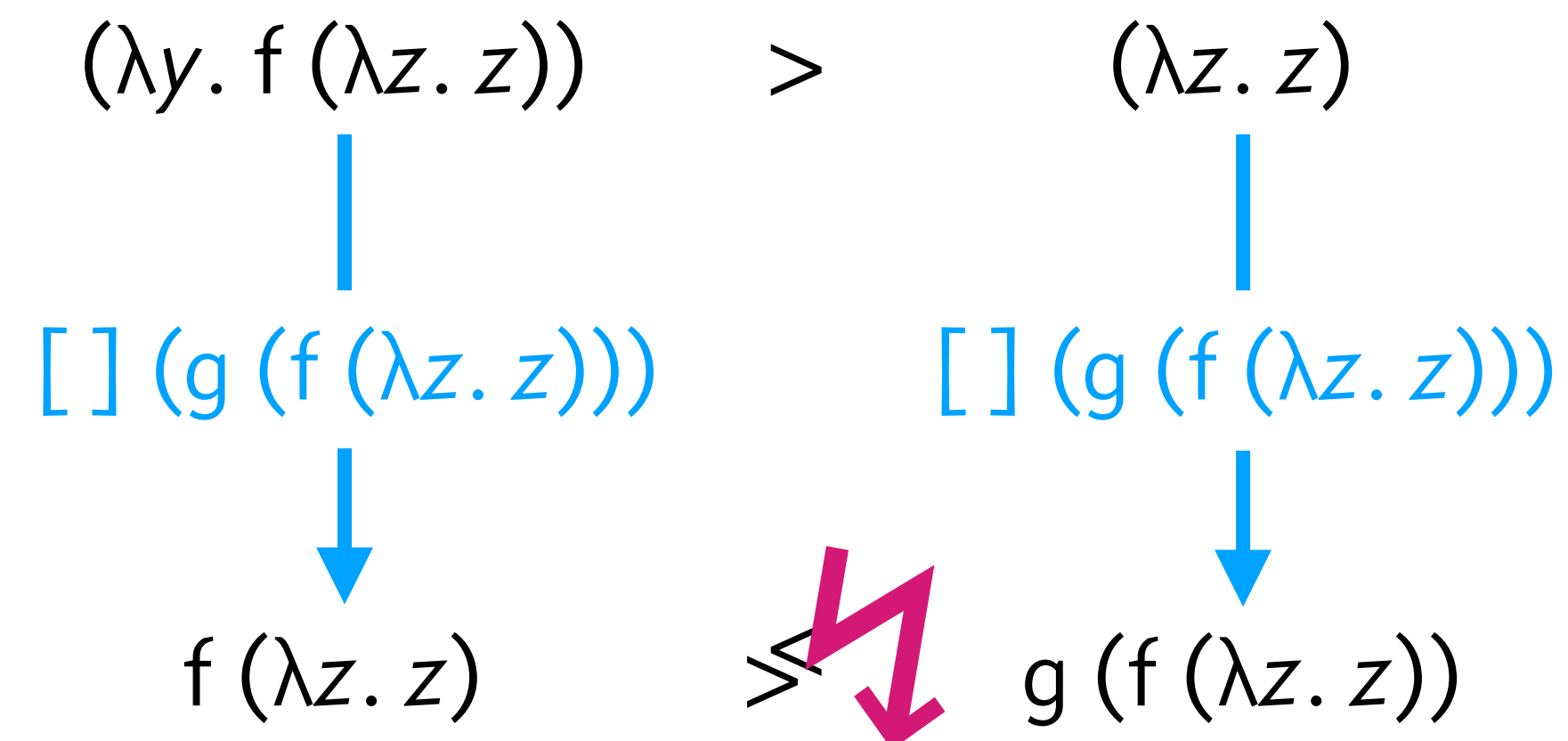
$$\begin{array}{ccc} (\lambda y. f (\lambda z. z)) & > & (\lambda z. z) \\ \downarrow & & \downarrow \\ [] (g (f (\lambda z. z))) & & [] (g (f (\lambda z. z))) \\ \downarrow & & \downarrow \\ f (\lambda z. z) & \lesssim & g (f (\lambda z. z)) \end{array}$$

Challenge 1: The Term Order

In first-order superposition:

- A subterm must be smaller than the whole term:
e.g. $c < g(c) < f(g(c))$
- Putting two terms in the same context should preserve the orientation:
e.g. if $b > a$ then $f(b, c) > f(a, c)$

Issue in higher-order:



Challenge 1: The Term Order

Solution:

- Weaken the second requirement: only **good contexts** must preserve the orientation
- The main superposition rule acts only on **good contexts**
- Compensate with an **extra calculus rule**

Challenge 1: The Term Order

Solution:

- Weaken the second requirement: only **good contexts** must preserve the orientation
- The main superposition rule acts only on **good contexts**
- Compensate with an **extra calculus rule**

Extra rule:

If the clause

$$C \vee t = t'$$

is contained in $\mathbb{F}$
and t, t' are functions,
then add the clause

$$C \vee t x = t' x$$

to $\mathbb{F}$

Example: The Term Order

$\mathbb{F}$

$$g = f$$

$$g a \neq f a$$

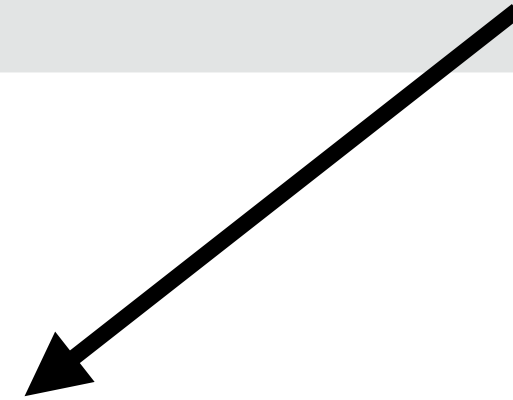
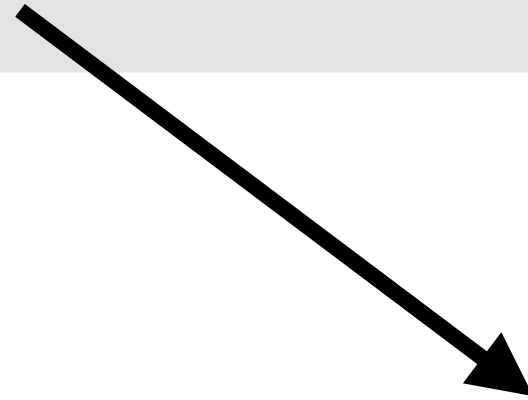
Example: The Term Order

$\mathbb{F}$

$$g = f$$

$$g a \neq f a$$

$$f a \neq f a$$



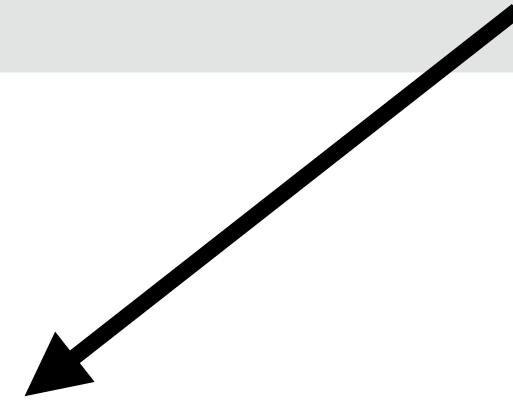
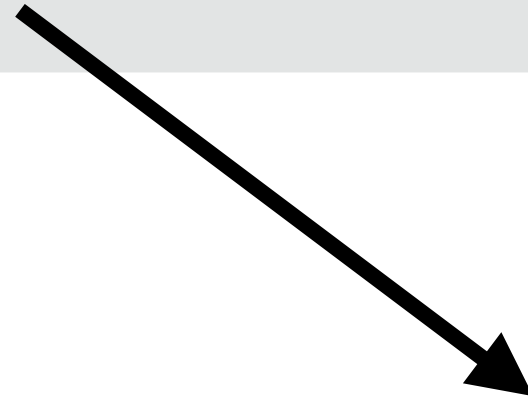
Example: The Term Order

$\mathbb{F}$

$$g = f$$

$$g a \neq f a$$

$$f a \neq f a$$



Example: The Term Order

$\mathcal{F}$

$g = f$

$g a \neq f a$

$f \neq f a$

disallowed because $[] a$ is not a good context

Example: The Term Order

$\mathbb{F}$

$$g = f$$

$$g a \neq f a$$

Example: The Term Order

$\mathbb{F}$

$$g = f$$

$$g\ a \neq f\ a$$

extra rule

$$g\ x = f\ x$$

Example: The Term Order

$\mathbb{F}$

$$g = f$$

$$g\ a \neq f\ a$$

$$g\ x = f\ x$$

Example: The Term Order

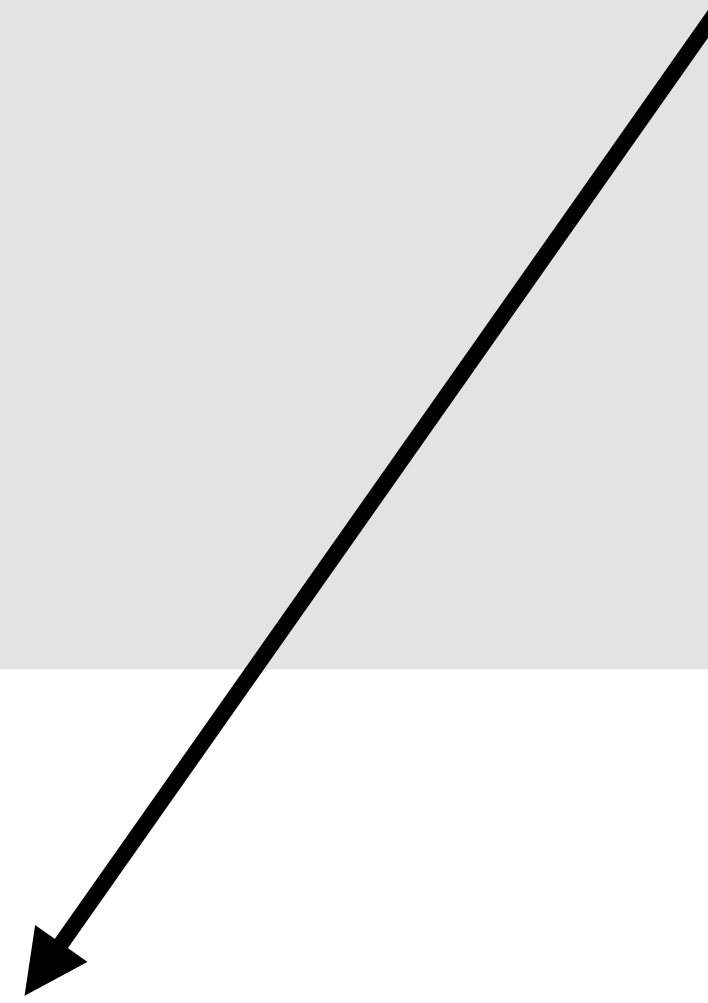
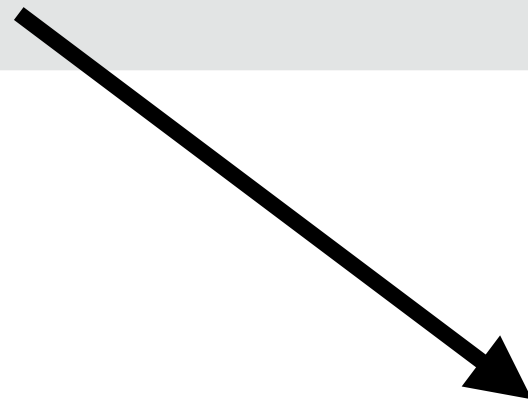
$\mathbb{F}$

$$g = f$$

$$g a \neq f a$$

$$g x = f x$$

$$f a \neq f a$$



Example: The Term Order

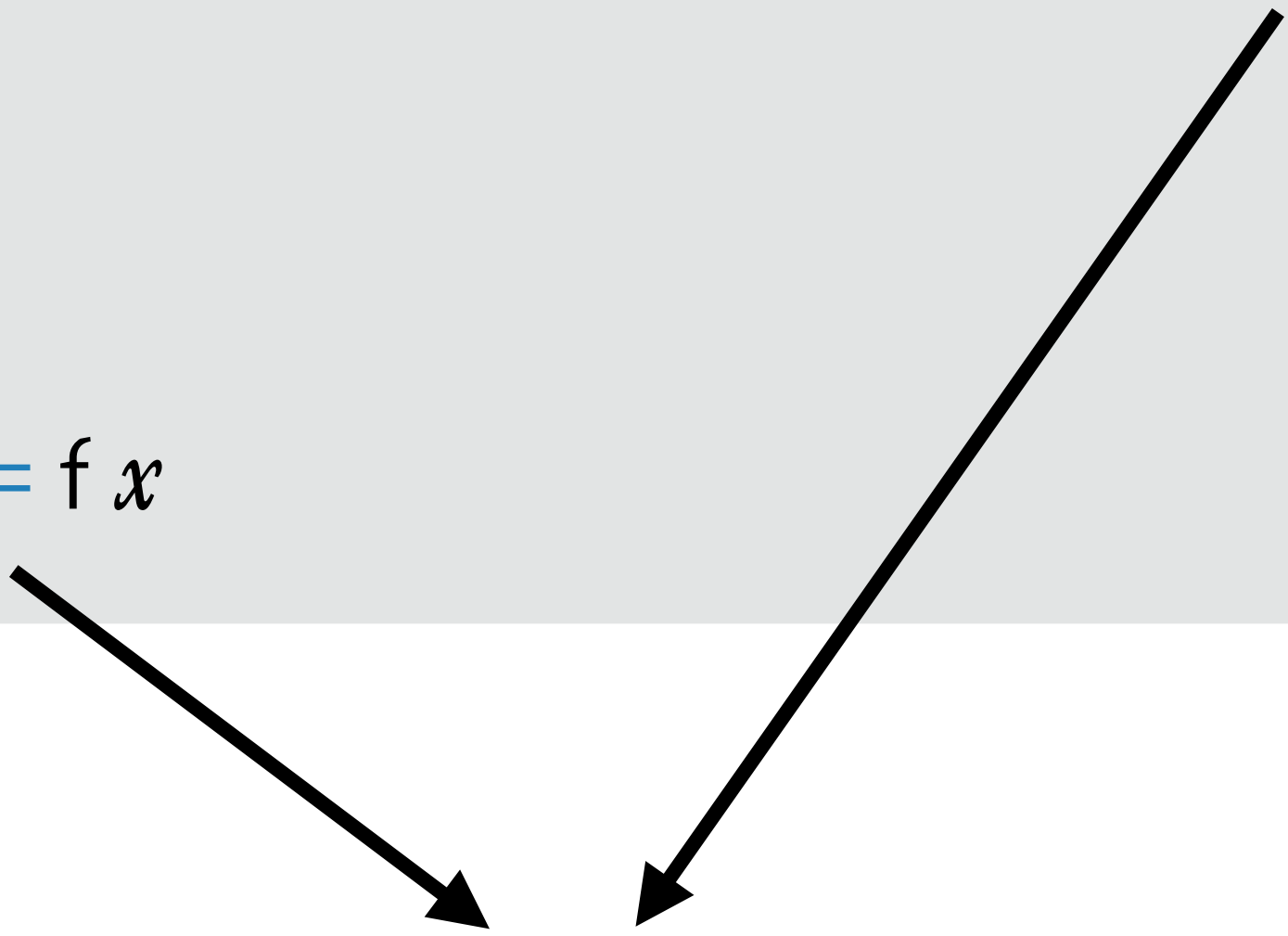
$\mathcal{F}$

$$g = f$$

$$g a \neq f a$$

$$g x = f x$$

$$f a \neq f a$$



Example: The Term Order

$\mathcal{F}$

$$g = f$$

$$g\ a \neq f\ a$$

$$g\ x = f\ x$$

$$f\ a \neq f\ a$$

Challenge 1 bis: The Redundancy Criterion

In first-order logic:

- A ground clause C is redundant w.r.t. ground $\mathbb{F}$ if $C_1, \dots, C_n \in \mathbb{F}$ are all smaller than C and they together entail C

Challenge 1 bis: The Redundancy Criterion

In first-order logic:

- A ground clause C is redundant w.r.t. ground $\mathbb{F}$ if $C_1, \dots, C_n \in \mathbb{F}$ are all smaller than C and they together entail C

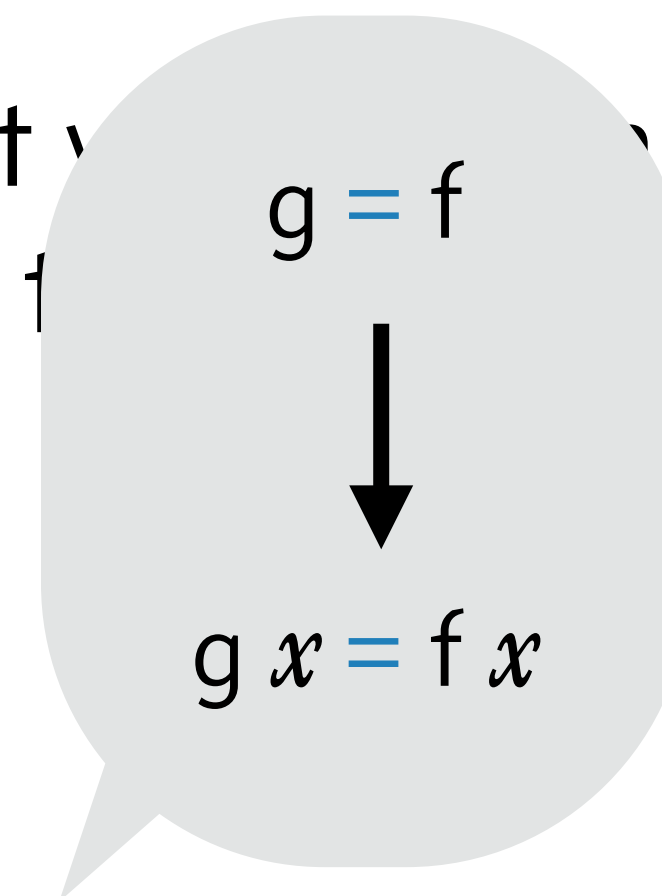
Higher-order issue:

- The conclusion of the extra inference rule would be redundant w.r.t. the premise

Challenge 1 bis: The Redundancy Criterion

In first-order logic:

- A ground clause C is redundant w.r.t. $\mathcal{F}$ if $C_1, \dots, C_n \in \mathcal{F}$ are all smaller than C and they together entail C



Higher-order issue:

- The conclusion of the extra inference rule would be redundant w.r.t. the premise

Challenge 1bis: The Redundancy Criterion

Solution:

- Use weaker notion of entailment

Examples:

- $b = a$ makes $f b = f a$ a redundant
- $g = f$ **does not** make $g a = f a$ a redundant

Challenge 1bis: The Redundancy Criterion

Solution:

- Use weaker notion of entailment

Examples:

- $b = a$ makes $f b = f a$ redundant
- $g_0 = f_0$ **does not** make $g_1 a_0 = f_1 a_0$ redundant

Nonground Redundancy Criterion

A nonground clause C is redundant w.r.t. nonground $\mathcal{F}$
if each clause in $\text{ground}(C)$ is redundant w.r.t. $\text{ground}(\mathcal{F})$

Nonground Redundancy Criterion

A nonground clause C is redundant w.r.t. nonground $\mathcal{F}$ if each clause in $\text{ground}(C)$ is redundant w.r.t. $\text{ground}(\mathcal{F})$

Examples:

- $b \ x = a$ makes $f(b \ x) = f$ a redundant
- $g = f$ **does not** make $g \ x = f \ x$ redundant

Example of a Redundancy Rule

Argument pruning:

If a clause of the form

$C(y)$

is contained in $\mathbb{F}$,
where one of y 's arguments is
computable from the others,
then **remove** the argument

Example of a Redundancy Rule

Argument pruning:

If a clause of the form

$$y \text{ b b } \neq y \text{ a a}$$

$C(y)$

is contained in $\mathbb{F}$,
where one of y 's arguments is
computable from the others,
then **remove** the argument

Example of a Redundancy Rule

Argument pruning:

If a clause of the form

$C(y)$

is contained in $\mathbb{F}$,
where one of y 's arguments is
computable from the others,
then **remove** the argument

~~$y b b \neq y a a$~~ $y b \neq y a$

Example of a Redundancy Rule

Argument pruning:

If a clause of the form

$$C(y)$$

is contained in $\mathbb{F}$,
where one of y 's arguments is
computable from the others,
then **remove** the argument

$$\cancel{y\ b\ b \neq y\ a\ a} \quad y\ b \neq y\ a$$

$$y\ a\ c \neq y\ a\ b$$

Example of a Redundancy Rule

Argument pruning:

If a clause of the form

$C(y)$

is contained in $\mathbb{F}$,
where one of y 's arguments is
computable from the others,
then **remove** the argument

~~$y b b \neq y a a$~~ $y b \neq y a$

~~$y a c \neq y a b$~~ $y c \neq y b$

Example of a Redundancy Rule

Argument pruning:

If a clause of the form

$C(y)$

is contained in $\mathbb{F}$,
where one of y 's arguments is
computable from the others,
then **remove** the argument

~~$y b b \neq y a a$~~ $y b \neq y a$

~~$y a c \neq y a b$~~ $y c \neq y b$

$y a c \neq z b d$

Example of a Redundancy Rule

Argument pruning:

If a clause of the form

$C(y)$

is contained in $\mathbb{F}$,
where one of y 's arguments is
computable from the others,
then **remove** the argument

~~$y b b \neq y a a$~~ $y b \neq y a$

~~$y a c \neq y a b$~~ $y c \neq y b$

~~$y a c \neq z b d$~~ $y \neq z$

Challenge 2: Unification

In first-order logic:

- Most general unifiers always exist and can be computed
e.g. unifying $f(a, y)$ with $f(x, b)$ yields the mgu $\{x \mapsto a, y \mapsto b\}$

Challenge 2: Unification

In first-order logic:

- Most general unifiers always exist and can be computed
e.g. unifying $f(a, y)$ with $f(x, b)$ yields the mgu $\{x \mapsto a, y \mapsto b\}$

In higher-order logic:

- A. Most general unifiers **do not** always exist
e.g. unifying $f(y\ a)$ and $y\ (f\ a)$ yields infinitely many unifiers
 $\{y \mapsto \lambda x. x\} \quad \{y \mapsto \lambda x. f\ x\} \quad \{y \mapsto \lambda x. f\ (f\ x)\} \quad \dots$
- B. Unification is undecidable
- C. Applied variables can hide positions where inferences should be made

Challenge 2: Unification

Solutions:

- A. Use a (possibly infinite) sequence of unifiers instead of mgu
- B. Interweave unification and inferences
- C. Introduce a special "fluid" version of the main inference

Example: Fluid Inference

$\mathbb{F}$

$f\ a = c$

$h\ (y\ b)\ (y\ a) \neq h\ (g\ (f\ b))\ (g\ c)$

Example: Fluid Inference

$\mathbb{F}$

$$f\ a = c$$

$$h\ (\gamma\ b)\ (\gamma\ a) \neq h\ (g\ (f\ b))\ (g\ c)$$



$$\{\gamma \mapsto \lambda x. g\ (f\ x)\}$$

$$h\ (g\ (f\ b))\ (g\ (f\ a)) \neq h\ (g\ (f\ b))\ (g\ c)$$

Example: Fluid Inference

$\mathbb{F}$

$$f\ a = c$$

$$h\ (y\ b)\ (y\ a) \neq h\ (g\ (f\ b))\ (g\ c)$$

$$\{y \mapsto \lambda x. g\ (f\ x)\}$$

$$h\ (g\ (f\ b))\ (g\ (f\ a)) \neq h\ (g\ (f\ b))\ (g\ c)$$

$$h\ (g\ (f\ b))\ (g\ c) \neq h\ (g\ (f\ b))\ (g\ c)$$

Example: Fluid Inference

$\mathbb{F}$

$$f\ a = c$$

$$h\ (y\ b)\ (y\ a) \neq h\ (g\ (f\ b))\ (g\ c)$$

$$\{y \mapsto \lambda x. g\ (f\ x)\}$$

$$h\ (g\ (f\ b))\ (g\ (f\ a)) \neq h\ (g\ (f\ b))\ (g\ c)$$

$$h\ (g\ (f\ b))\ (g\ c) \neq h\ (g\ (f\ b))\ (g\ c)$$

Example: Fluid Inference

$\mathbb{F}$

$f\ a = c$

$h\ (\gamma\ b)\ (\gamma\ a) \neq h\ (g\ (f\ b))\ (g\ c)$

$\{\gamma \mapsto \lambda x. g\ (f\ x)\}$

$h\ (g\ (f\ b))\ (g\ (f\ a)) \neq h\ (g\ (f\ b))\ (g\ c)$

$h\ (g\ (f\ b))\ (g\ c) \neq h\ (g\ (f\ b))\ (g\ c)$

Example: Fluid Inference

$\mathbb{F}$

$$z(f\ a) = z\ c$$

$$h(y\ b)\ (y\ a) \neq h(g\ (f\ b))\ (g\ c)$$

Example: Fluid Inference

F

$z(f\ a) = z\ c$

$h(y\ b)\ (y\ a) \neq h(g(f\ b))\ (g\ c)$

$z(f\ a)$ is unified
with $y\ a$

Example: Fluid Inference

F

$$z(f\ a) = z\ c$$

$$h(y\ b)\ (y\ a) \neq h(g(f\ b))\ (g\ c)$$

$z(f\ a)$ is unified
with $y\ a$
 $\{z \mapsto g, y \mapsto \lambda x. g(f\ x)\}$

$$h(g(f\ b))\ (g\ c) \neq h(g(f\ b))\ (g\ c)$$

Challenge 3: Boolean Expressions

In first-order logic:

- Terms and formulas are syntactically distinct
- Clausification is simple and focuses on the outer skeleton

Challenge 3: Boolean Expressions

In first-order logic:

- Terms and formulas are syntactically distinct
- Clausification is simple and focuses on the outer skeleton

Higher-order issues:

- A. Formulas can appear nested in terms, including under λ 's
e.g. $(\lambda x. \text{if } \exists y. p \ x \ y \text{ then } a \text{ else } b)$
- B. We cannot perform clausification entirely in preprocessing

Challenge 3: Boolean Expressions

Solution:

- We introduce dedicated inference rules to clausify dynamically

Soundness of λ -Superposition

The inference rules are easy to show sound

In particular, the extra rule is justified by **argument congruence**:
if $\llbracket g \rrbracket^J = \llbracket f \rrbracket^J$, then $\llbracket g \ a \rrbracket^J = \llbracket f \ a \rrbracket^J$ for any a

Completeness of λ -Superposition

If the clause set $\mathcal{F}$ is initially unsatisfiable*
and inferences are performed fairly,
then $\mathcal{F}$ will eventually contain $\perp$

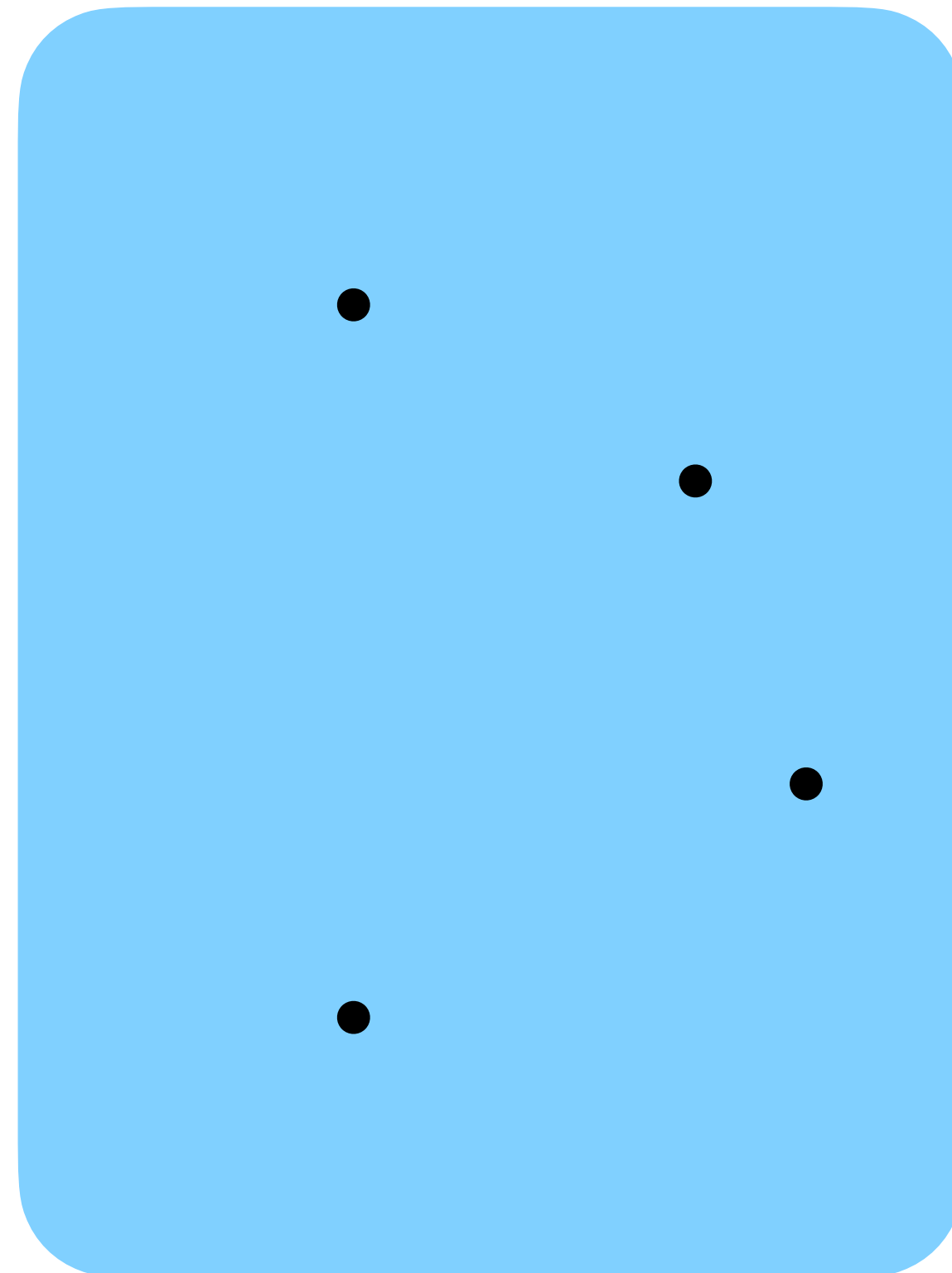
Completeness of λ -Superposition

If the clause set $\mathcal{F}$ is initially unsatisfiable*
and inferences are performed fairly,
then $\mathcal{F}$ will eventually contain $\perp$

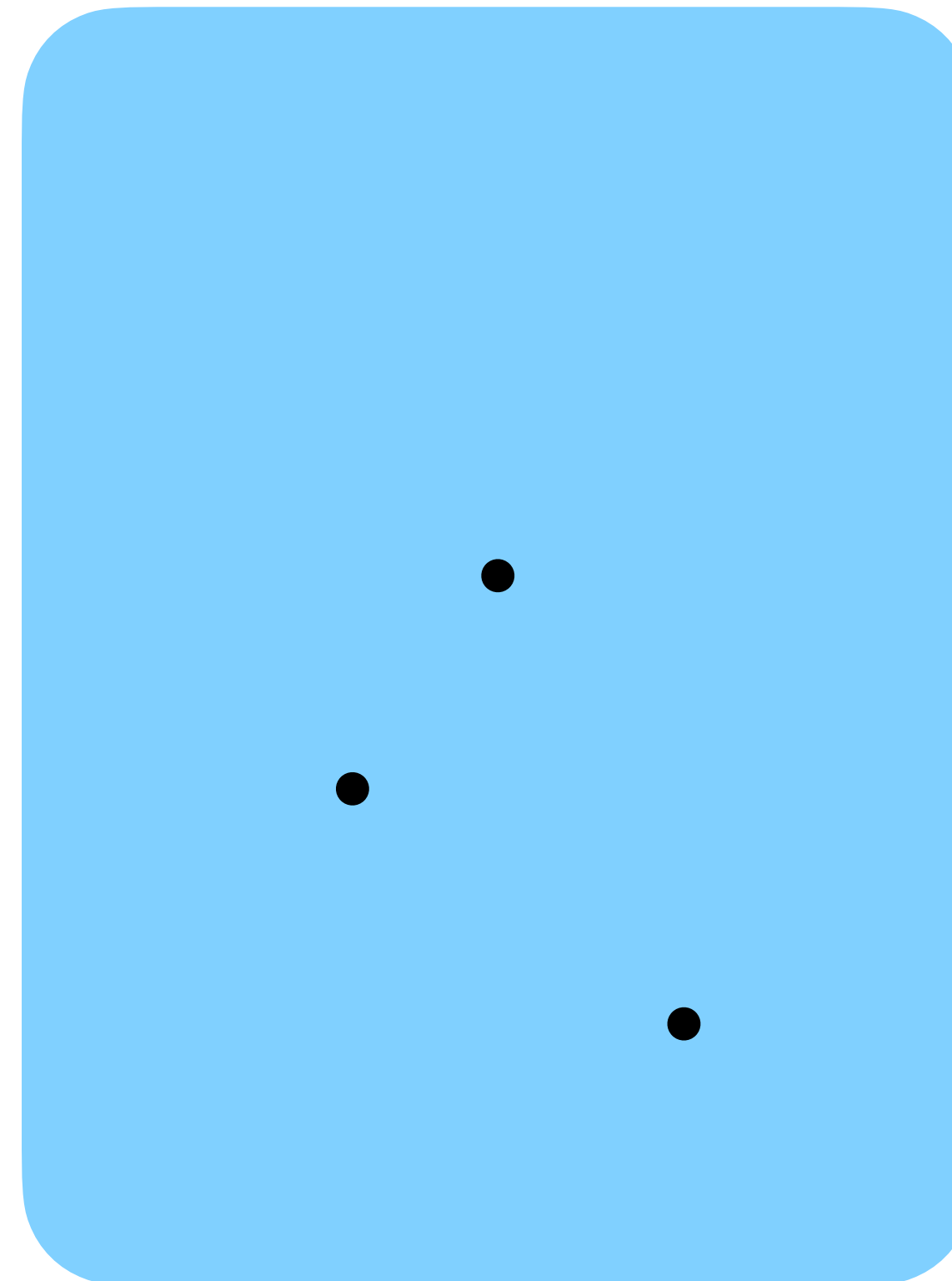
* with respect to the so-called Henkin semantics

Standard Saturation Loop

Passive

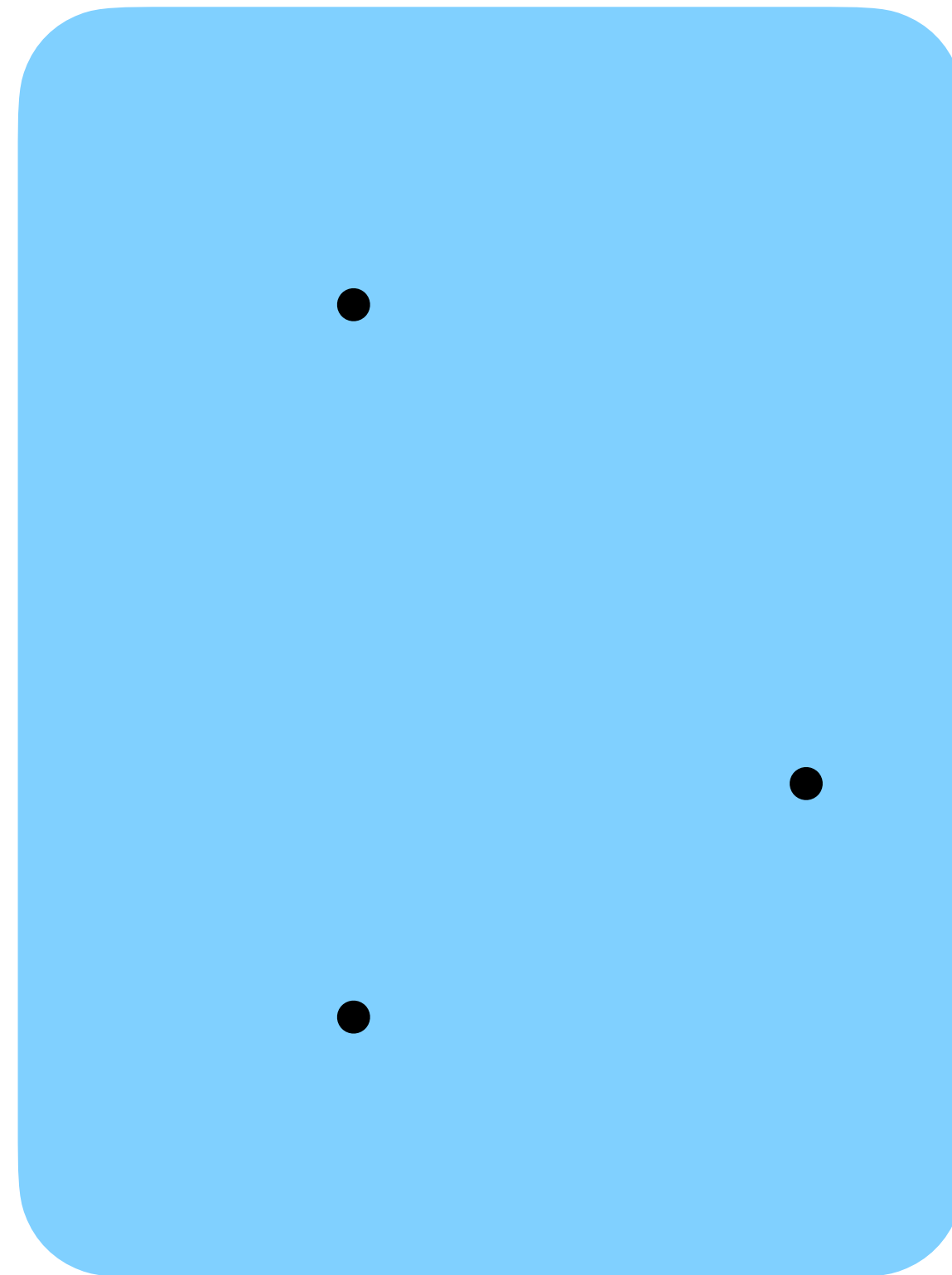


Active

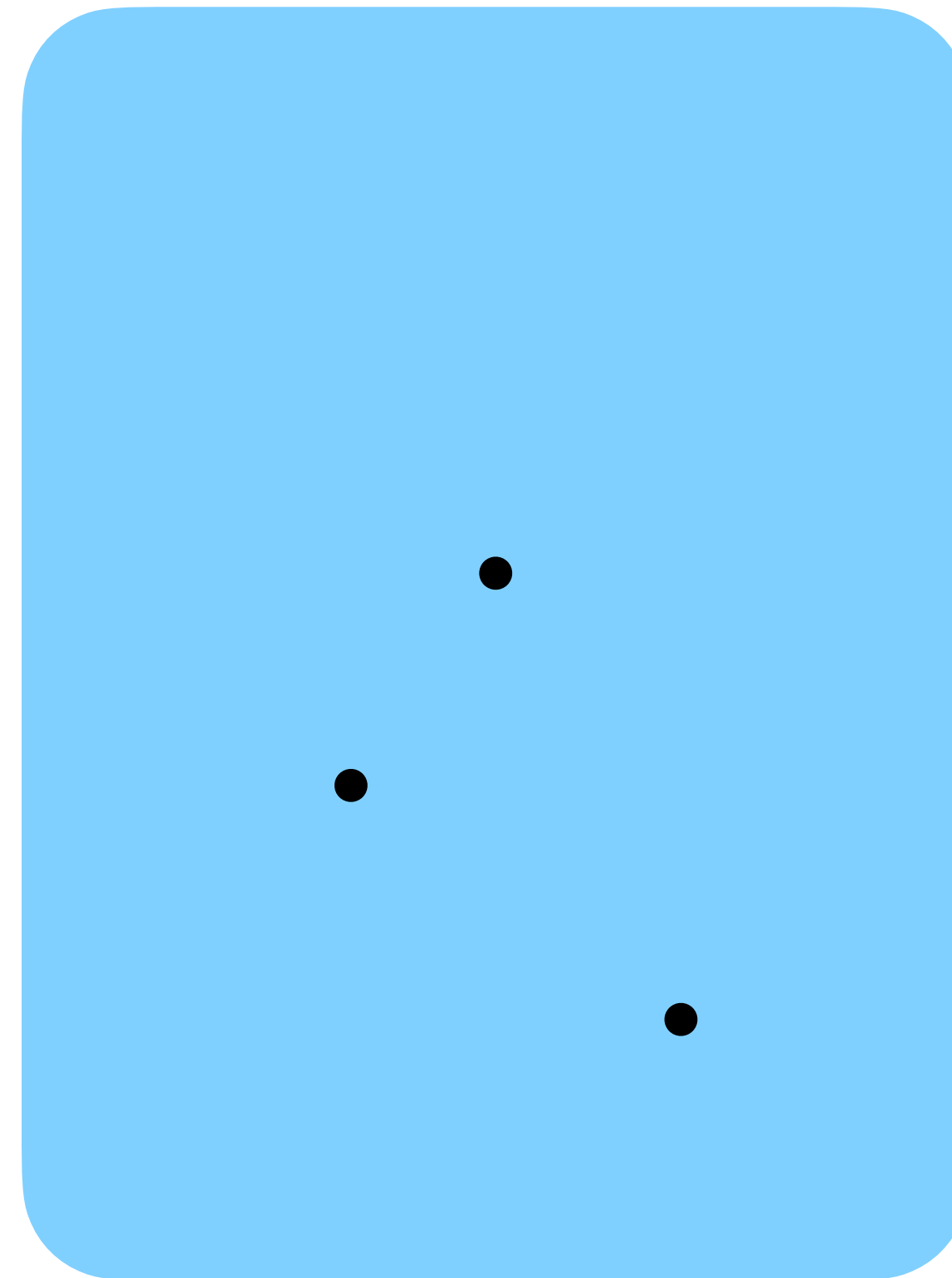


Standard Saturation Loop

Passive

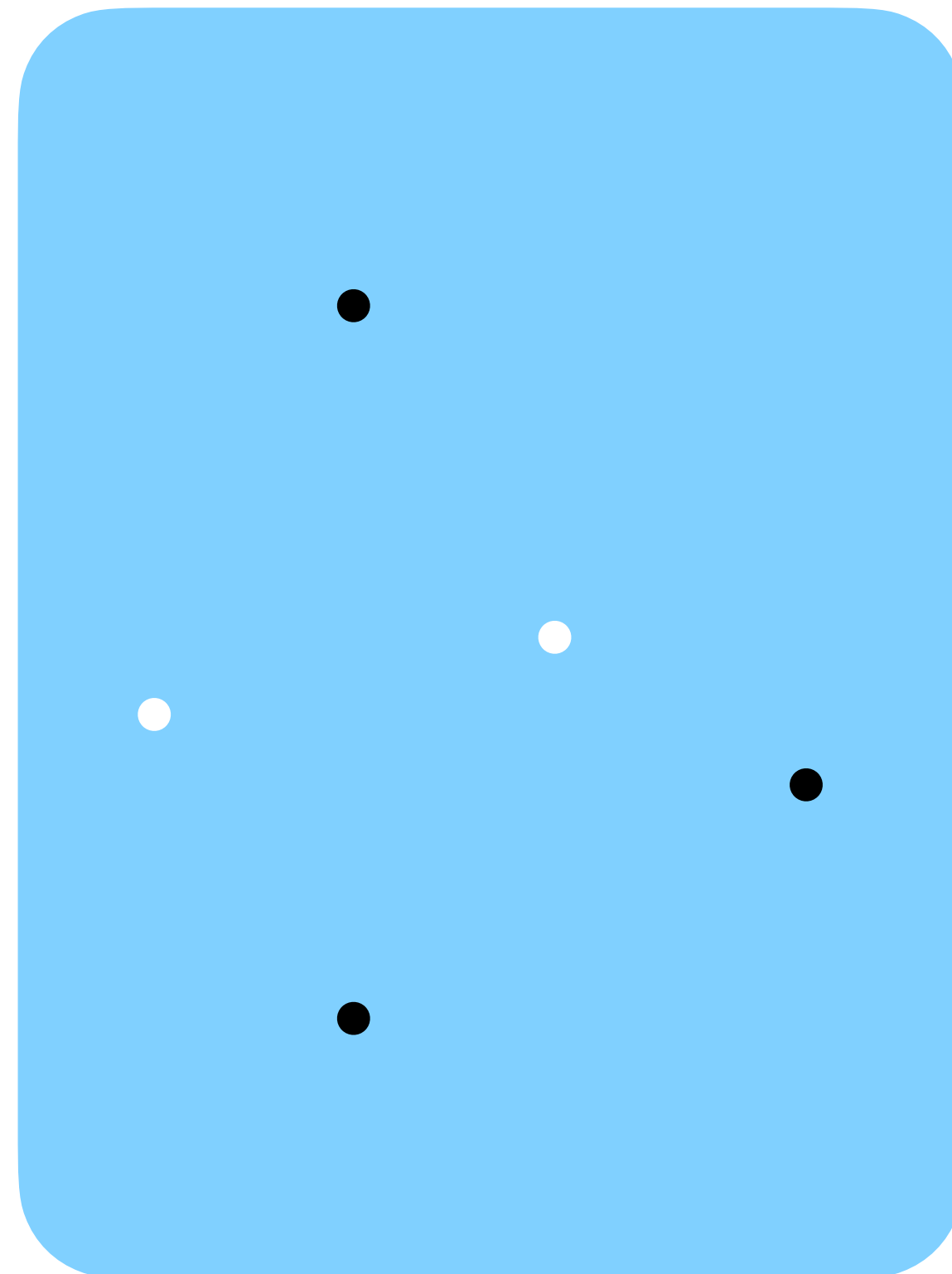


Active

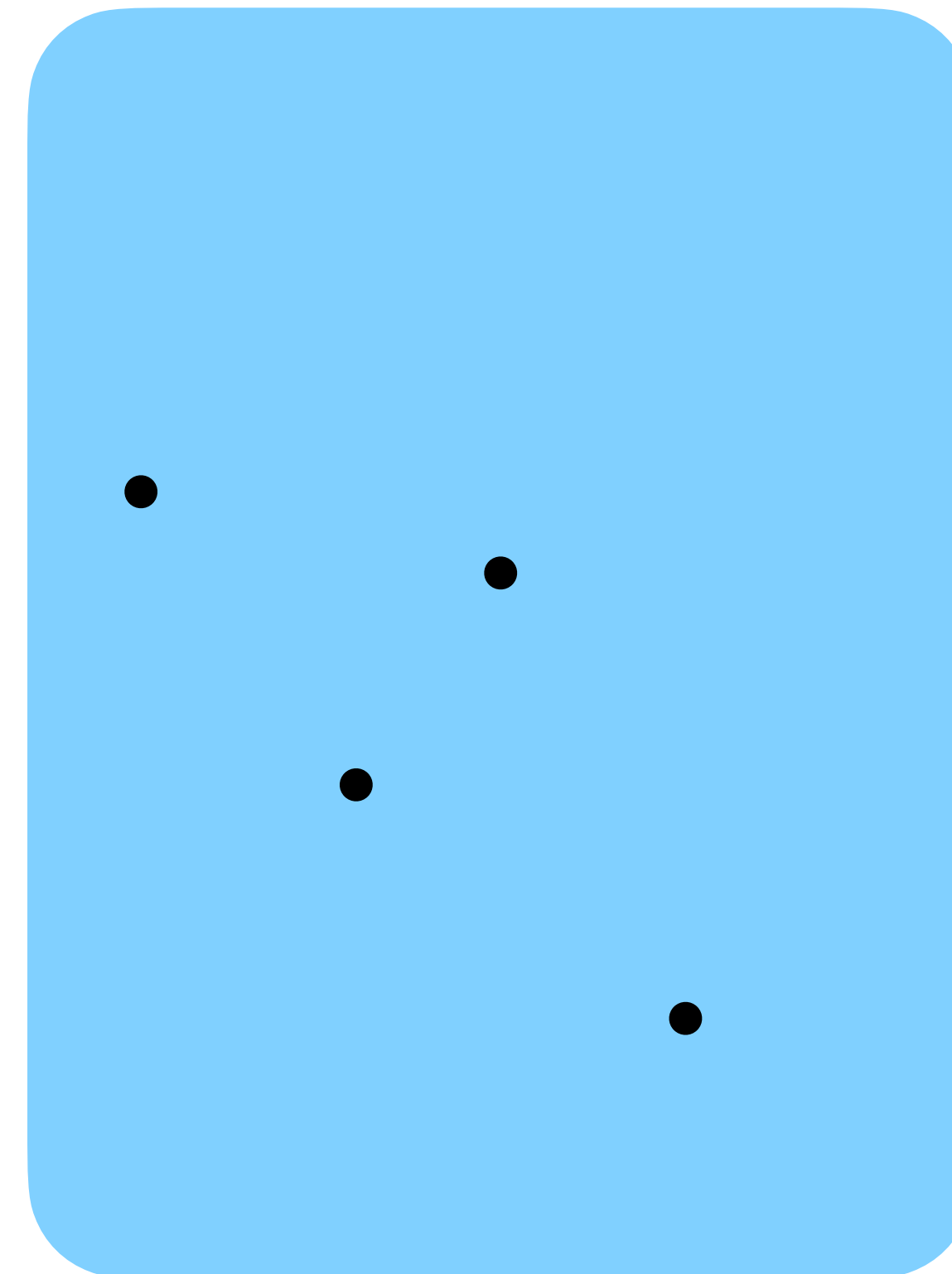


Standard Saturation Loop

Passive

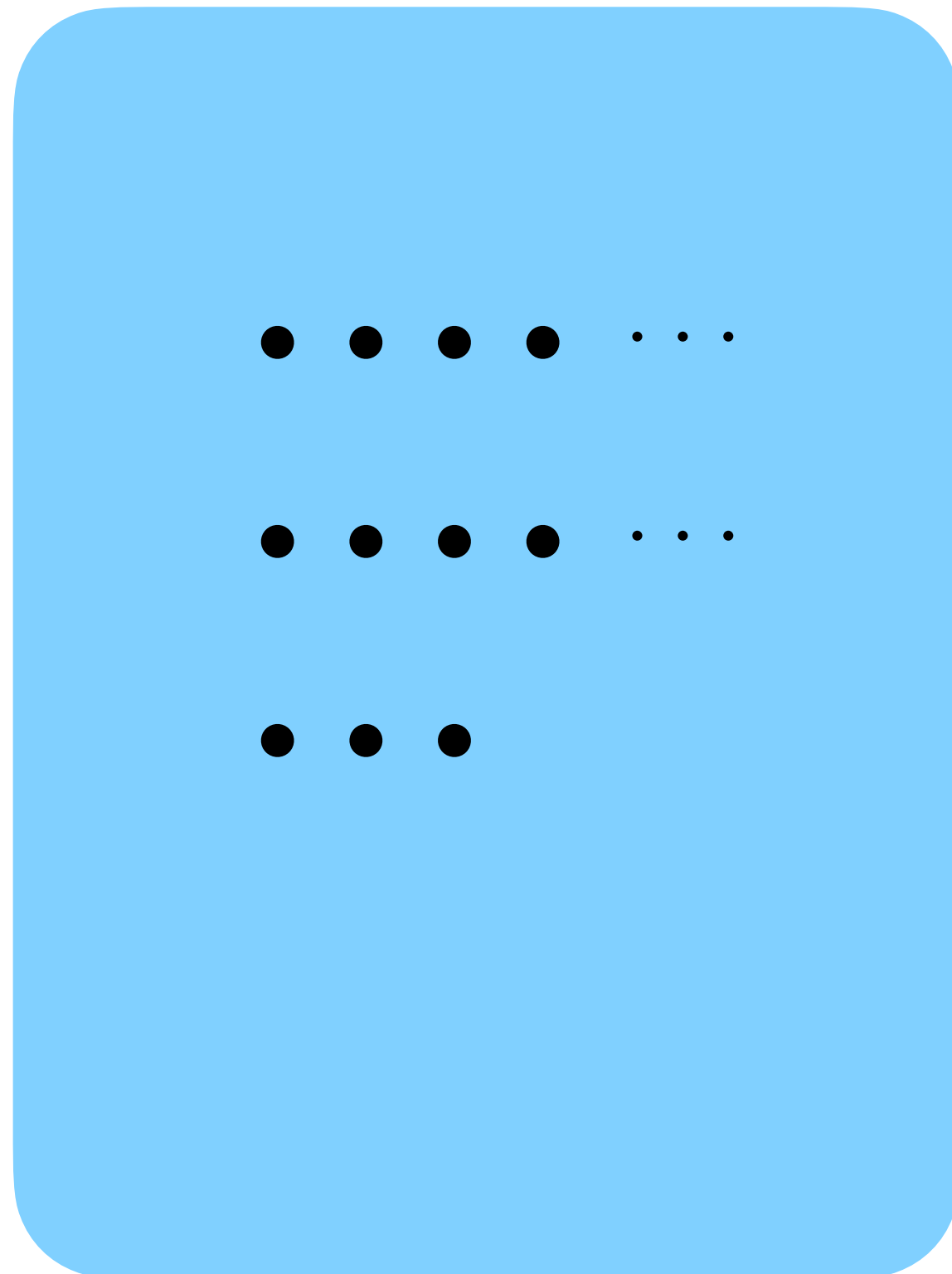


Active

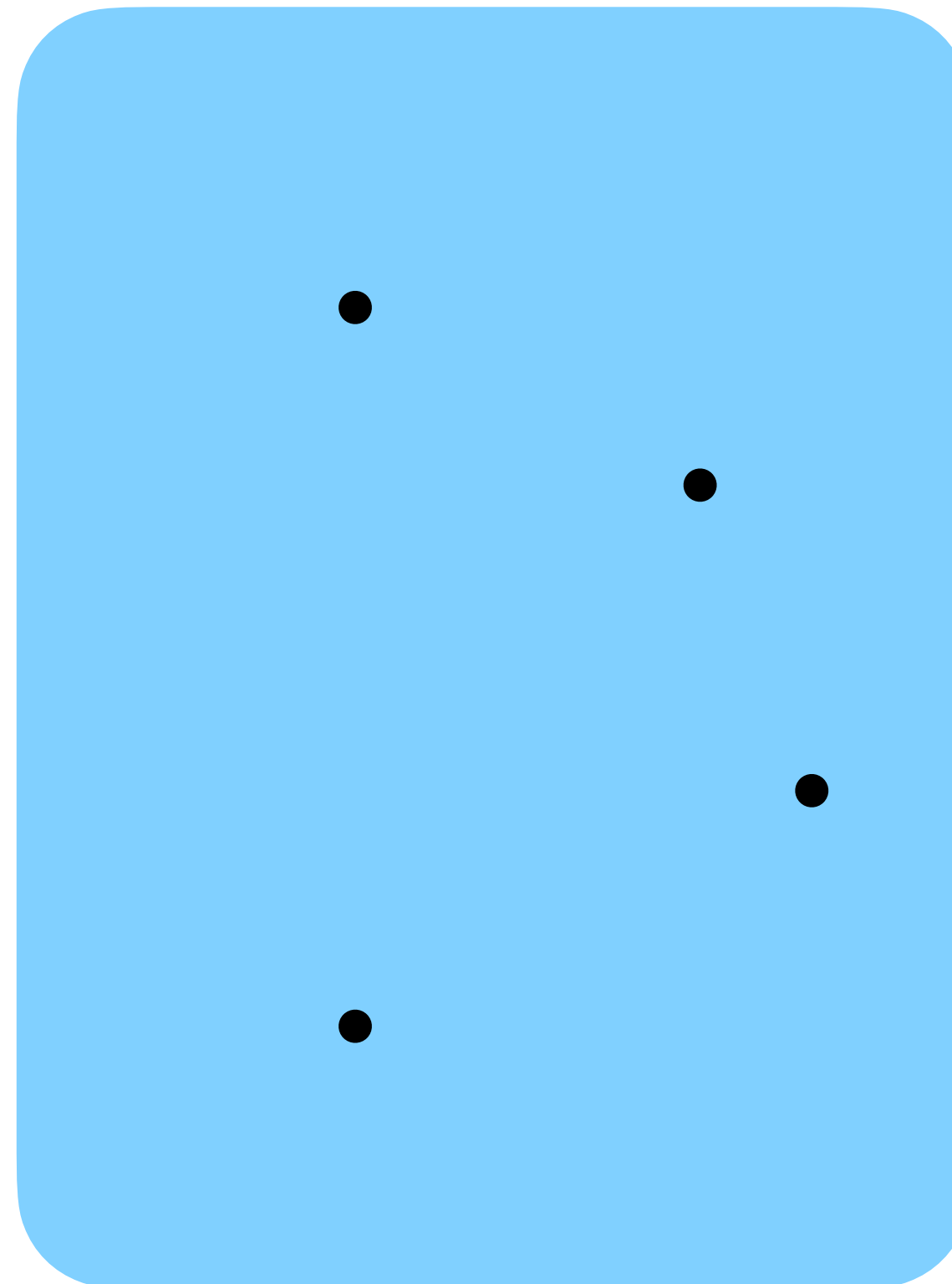


Generalized Saturation Loop

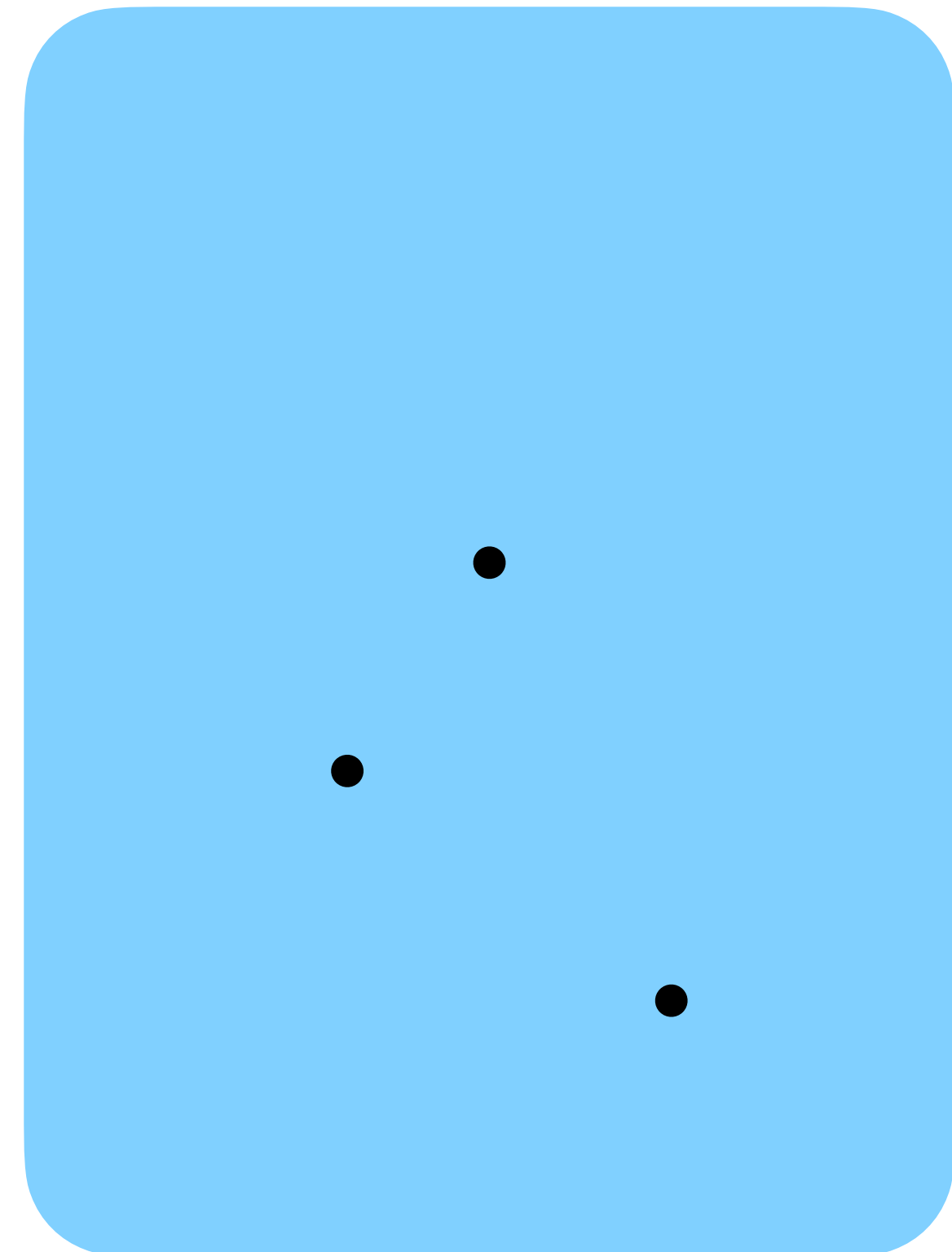
Streams



Passive

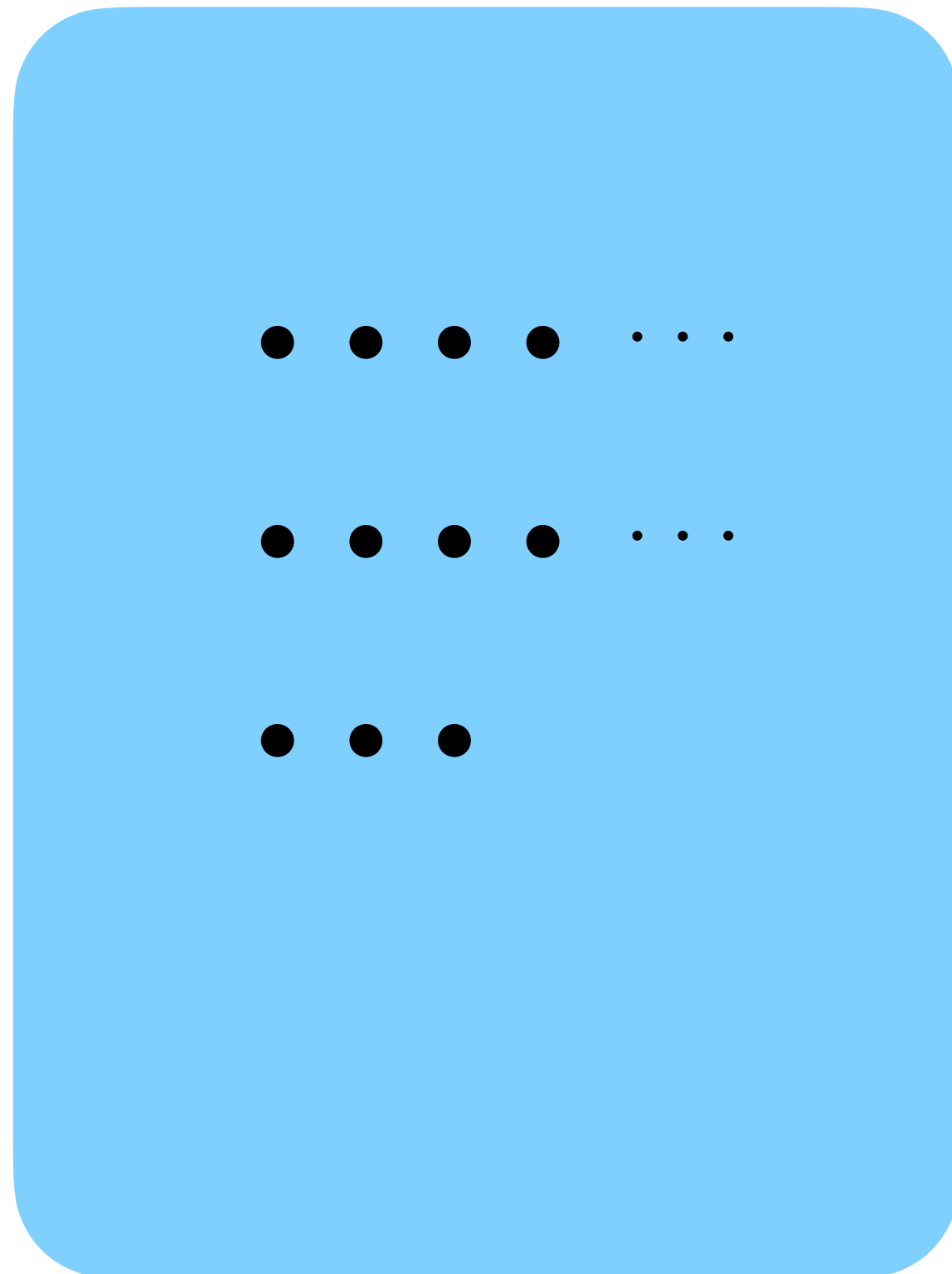


Active

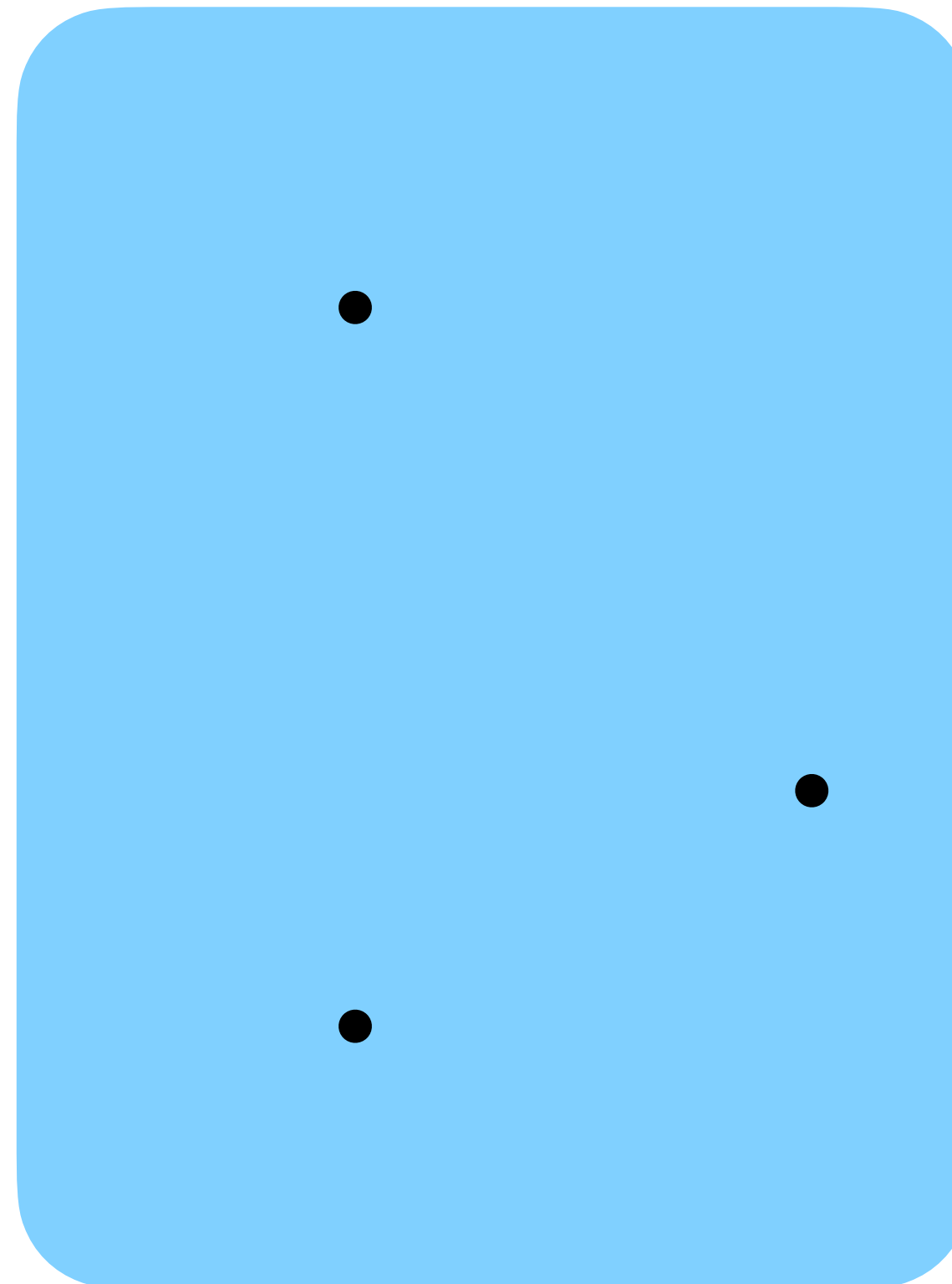


Generalized Saturation Loop

Streams

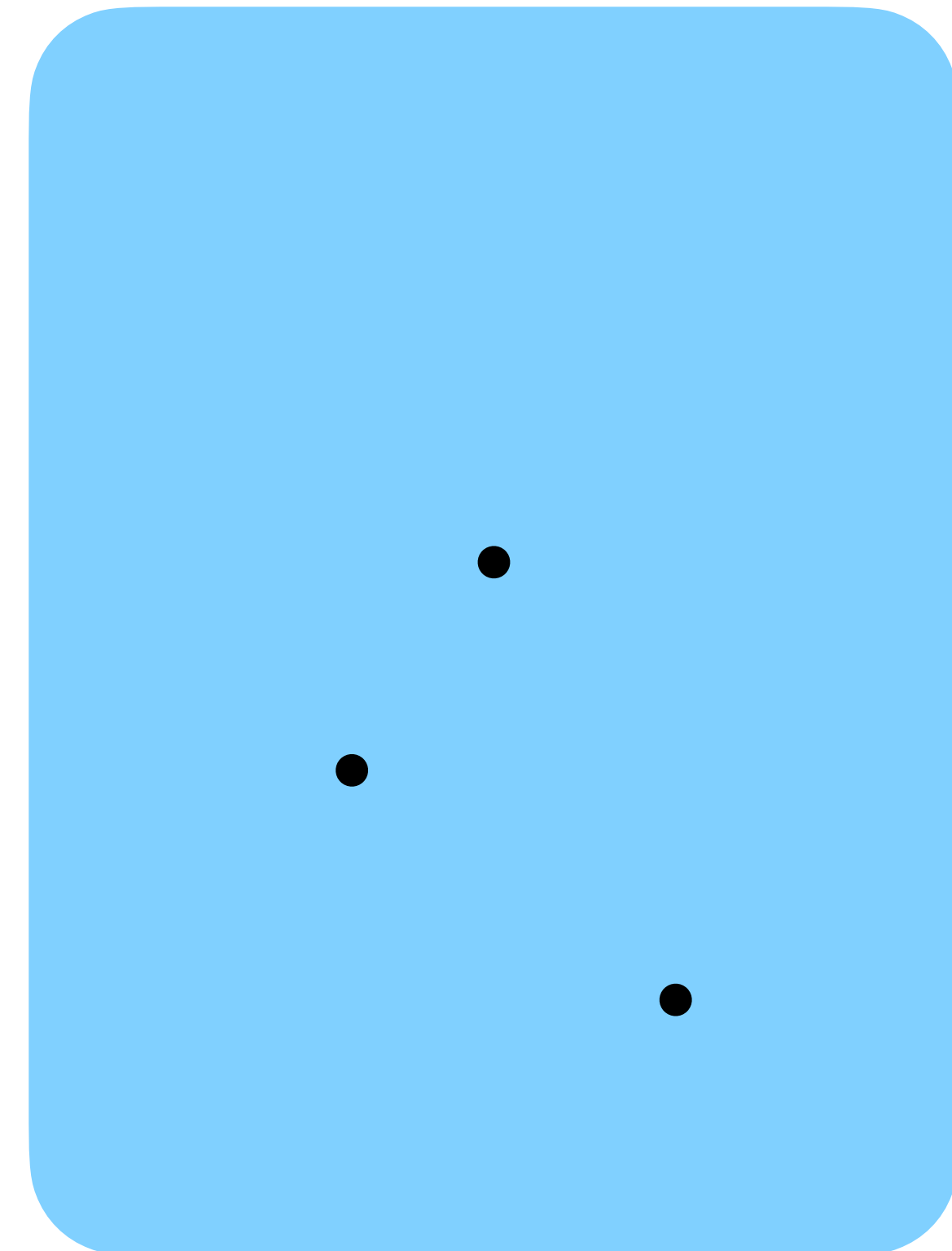


Passive



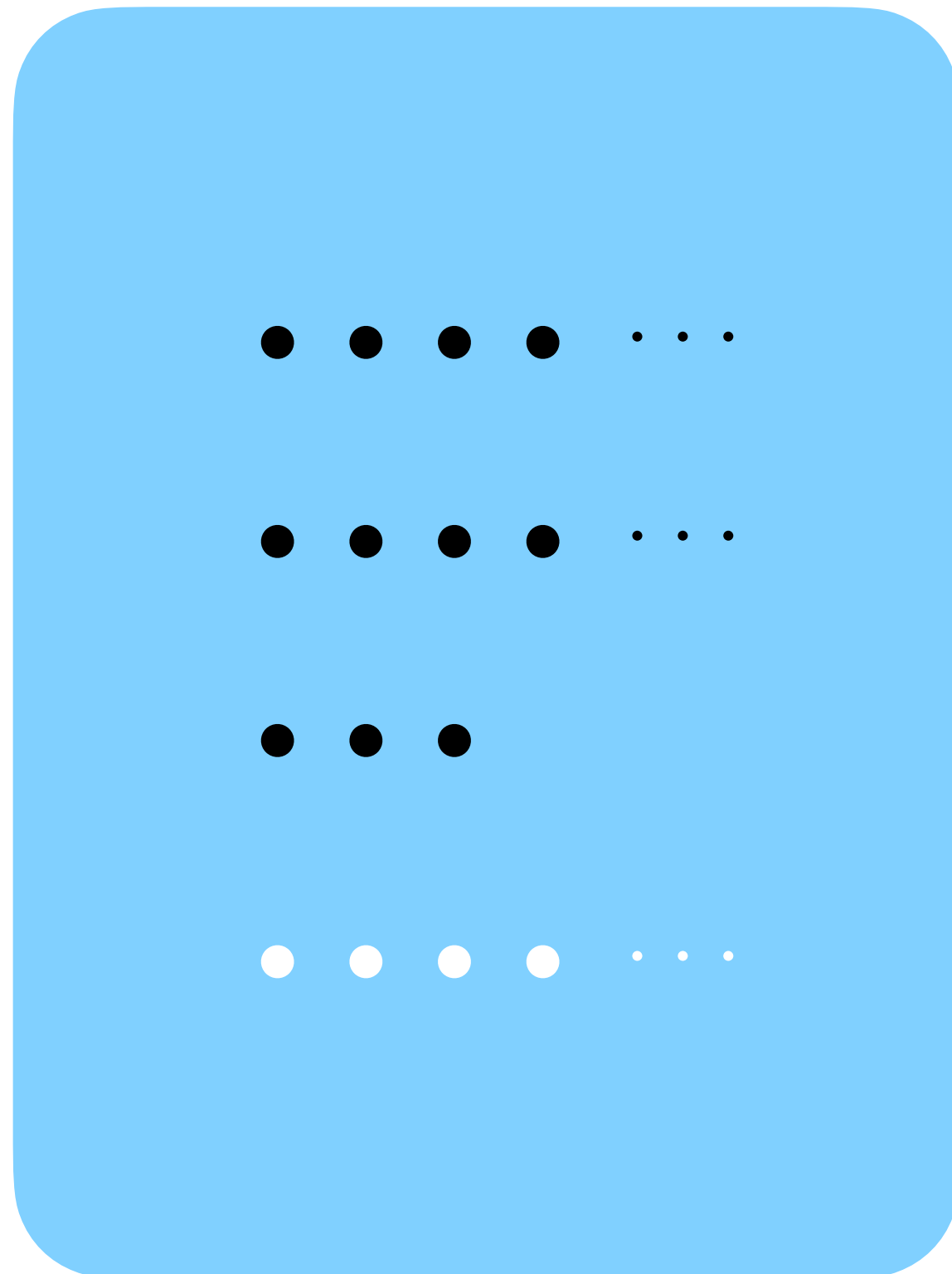
•

Active

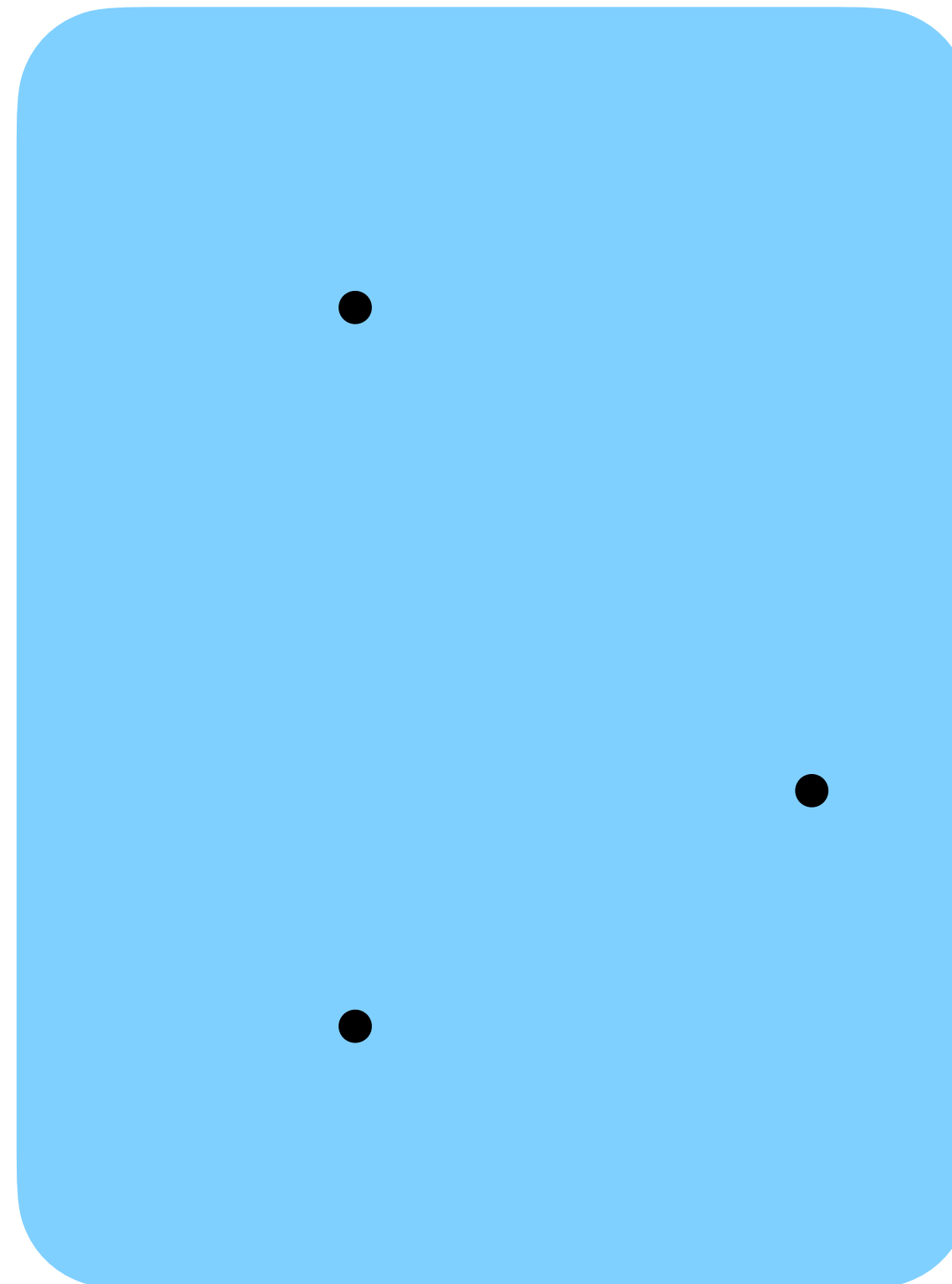


Generalized Saturation Loop

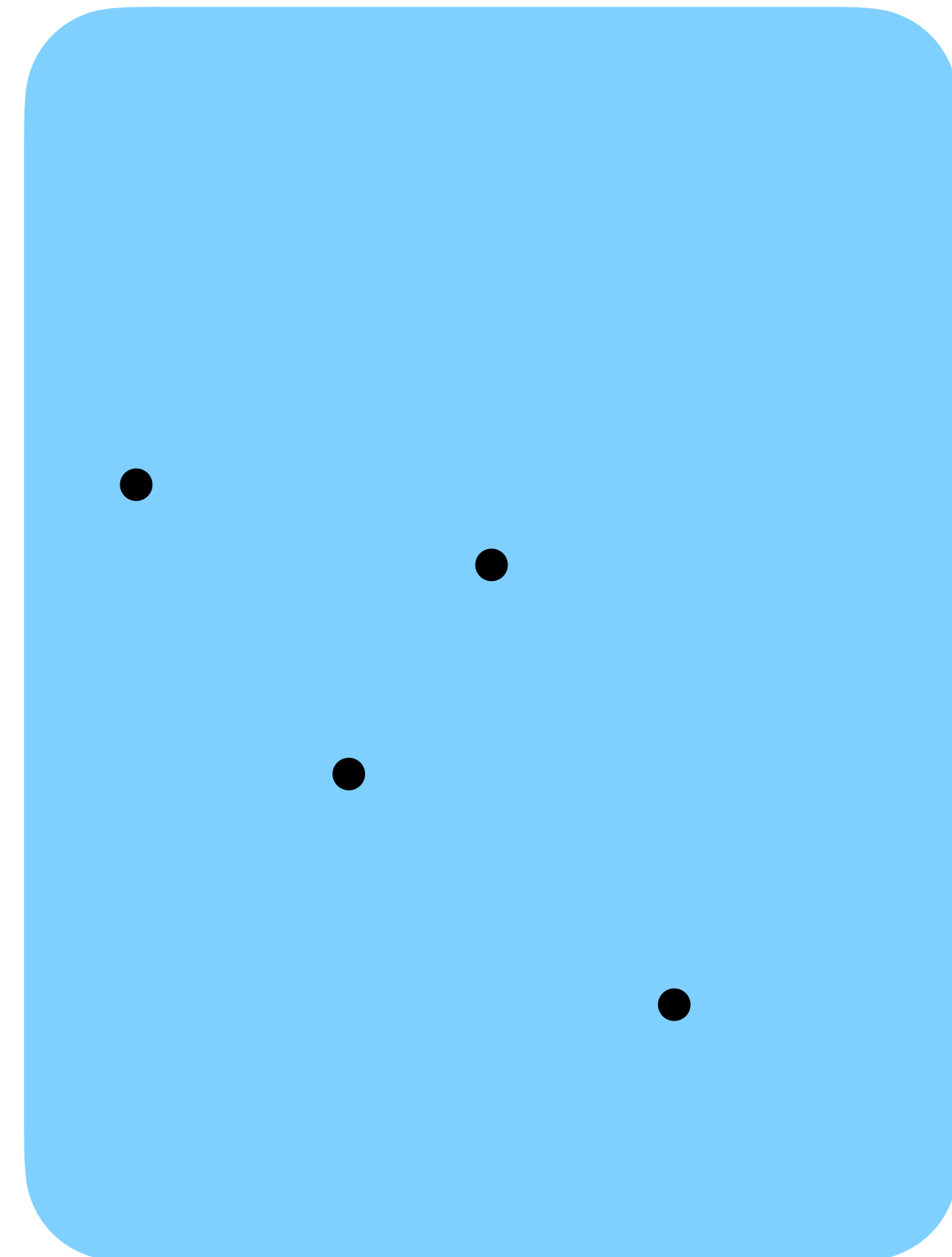
Streams



Passive



Active



References

Superposition with Lambdas

A. Bentkamp, J. Blanchette, S. Tourret, P. Vukmirović, and U. Waldmann
Journal of Automated Reasoning 65(7), 2021

Superposition for Higher-Order Logic

A. Bentkamp, J. Blanchette, S. Tourret, and P. Vukmirović
Journal of Automated Reasoning 67, article number 10, 2023

Mechanical Mathematicians

A. Bentkamp, J. Blanchette, V. Nummelin, S. Tourret, P. Vukmirović, and U. Waldmann
Communications of the ACM 66(4), 2023

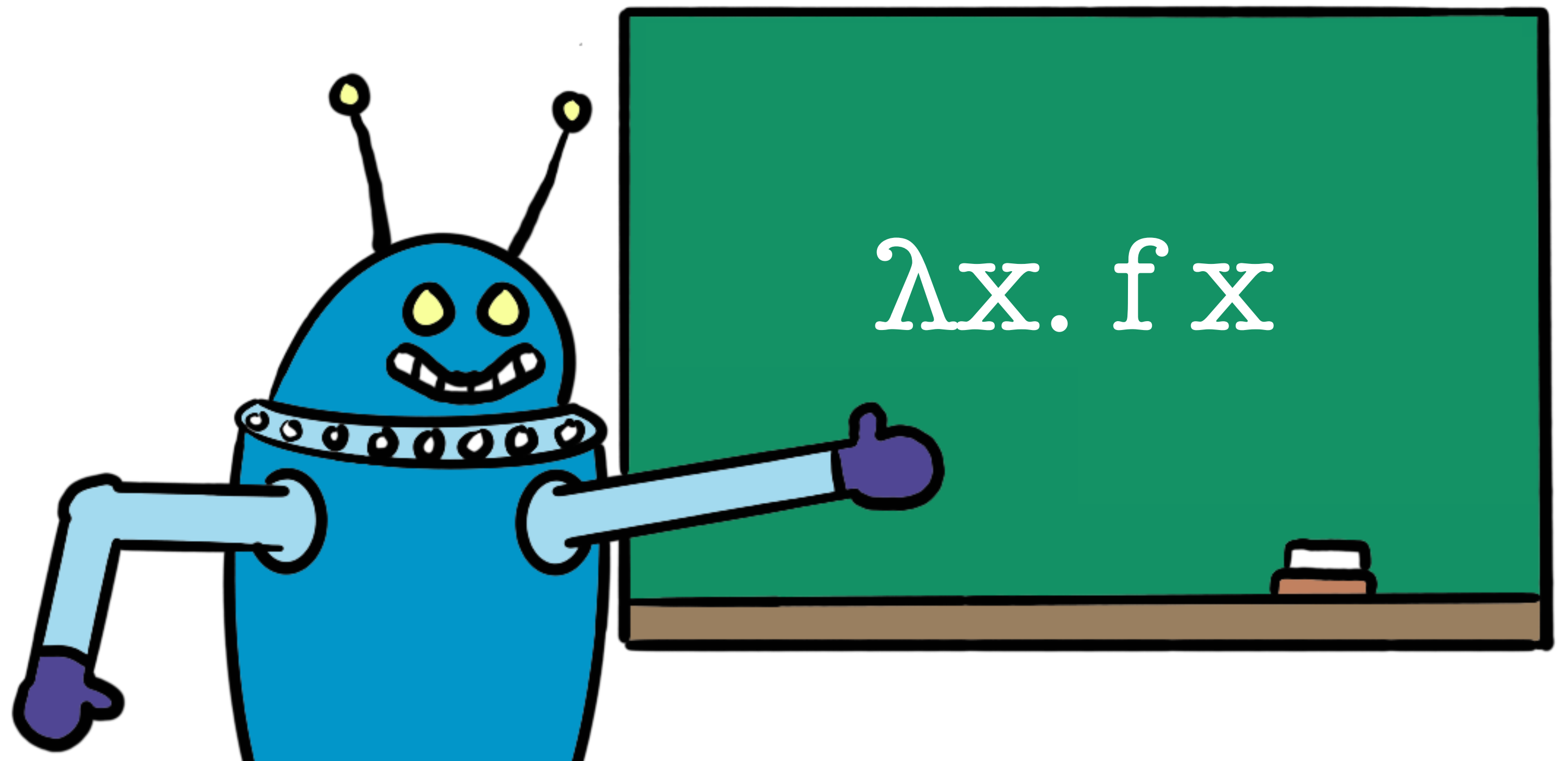
Provers and Solvers

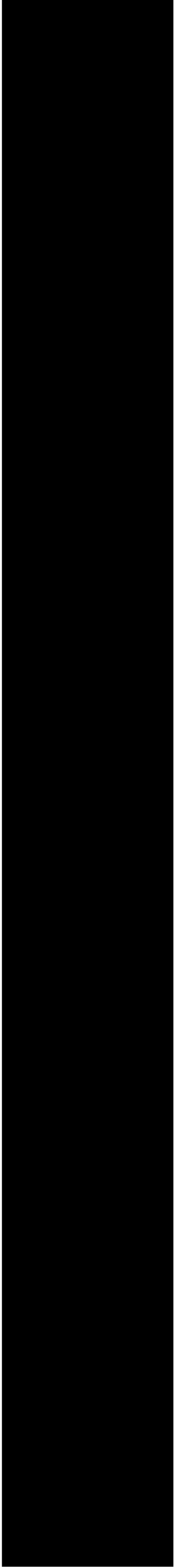
Lecture 3: λ -Superposition

Jasmin Blanchette

LMU Munich

Partly based on slides by
Alexander Bentkamp





Blah

